

UNIVERSIDAD DE CASTILLA-LA MANCHA
GRADO EN INGENIERÍA INFORMÁTICA

Trabajo Fin de Grado

**Aplicación móvil para la gestión digital del
armario**

Mobile app for digital wardrobe management

Victoria López González

Julio, 2026



Facultad de
**Ciencias Sociales y
Tecnologías de la Información**
Talavera de la Reina. UCLM

UNIVERSIDAD DE CASTILLA-LA MANCHA
Tecnologías y Sistemas de Información

GRADO EN INGENIERÍA INFORMÁTICA

**Mención en
SISTEMAS DE INFORMACIÓN**

Trabajo Fin de Grado

Aplicación móvil para la gestión digital del armario

Autoría: Victoria López González

Tutoría: Ana Rubio Ruiz

Cotutoría: Rubén Cantarero Navarro

Julio, 2026

Victoria López González

Talavera de la Reina – España

© 2026 Victoria López González

Permission is granted to copy, distribute and/or modify this document under the terms of the GNU Free Documentation License, Version 1.3 or any later version published by the Free Software Foundation; with no Invariant Sections, no Front-Cover Texts, and no Back-Cover Texts. A copy of the license is included in the section entitled "GNU Free Documentation License".

Se permite la copia, distribución y/o modificación de este documento bajo los términos de la Licencia de Documentación Libre GNU, versión 1.3 o cualquier versión posterior publicada por la *Free Software Foundation*; sin secciones invariantes. Una copia de esta licencia esta incluida en el apéndice titulado «GNU Free Documentation License».

Muchos de los nombres usados por las compañías para diferenciar sus productos y servicios son reclamados como marcas registradas. Allí donde estos nombres aparezcan en este documento, y cuando la persona autora haya sido informada de esas marcas registradas, los nombres estarán escritos en mayúsculas o como nombres propios.

TRIBUNAL:**Presidencia:****Vocal:****Secretaría:****FECHA DE DEFENSA:****CALIFICACIÓN:****PRESIDENCIA****VOCAL****SECRETARÍA**

Fdo.:

Fdo.:

Fdo.:

Resumen

En la actualidad, la gestión eficiente del vestuario personal presenta desafíos cotidianos que abarcan desde la optimización del espacio físico de almacenamiento hasta la toma de decisiones sobre la vestimenta en función de las condiciones meteorológicas, el tipo de evento y el color de las prendas. Para abordar esta problemática, el presente Trabajo de Fin de Grado detalla el diseño, desarrollo e implementación de *AmarioVirtual*, una aplicación móvil desarrollada bajo la integración metodológica de Scrum y OpenUp. A nivel tecnológico, la solución adopta una arquitectura distribuida que combina una interfaz multiplataforma construida con *React Native* y *TypeScript*, respaldada por la robustez del modelo *Backend as a Service* (BaaS) de Supabase (PostgreSQL) para la persistencia relacional y el almacenamiento seguro. El núcleo diferencial del proyecto radica en la delegación de las tareas computacionales de mayor coste a un microservicio independiente desarrollado en Python, el cual integra modelos de Inteligencia Artificial (*Deep Learning*) para automatizar la digitalización del catálogo mediante la eliminación semántica de fondos fotográficos y el Reconocimiento Óptico de Caracteres (*OCR*) de las etiquetas de lavado.

A partir de este inventario digitalizado, el sistema implementa una lógica de negocio avanzada compuesta por tres motores principales que transforman la aplicación en un asistente personal automatizado. En primer lugar, un algoritmo de recomendación sensible al contexto cruza la ropa limpia disponible con variables meteorológicas en tiempo real (mediante la API de *OpenWeatherMap*) para generar conjuntos completamente adaptados al clima. En segundo lugar, un sistema experto evalúa la composición y tipología de cada prenda para sugerir su organización espacial óptima dentro del armario físico. Por último, un planificador que agrupa automáticamente la ropa sucia en coladas seguras, aplicando restricciones simultáneas de colorimetría, fragilidad de los tejidos y temperaturas máxima. En definitiva, esta sinergia de tecnologías modernas y reglas heurísticas logra trascender el concepto de invierno estático, optimizando de forma integral el tiempo del usuario y la preservación de su vestuario.

Abstract

Currently, the efficient management of a personal wardrobe presents daily challenges ranging from the optimization of physical storage space to decision-making regarding clothing based on weather conditions, event type, and garment color. To address this issue, this Bachelor's Thesis details the design, development, and implementation of *ArmarioVirtual*, a mobile application developed under the methodological integration of Scrum and OpenUP. Technologically, the solution adopts a distributed architecture that combines a cross-platform interface built with *React Native* and *TypeScript*, backed by the robustness of Supabase's *Backend as a Service* (BaaS) model (PostgreSQL) for relational persistence and secure storage. The differential core of the project lies in delegating the most computationally expensive tasks to an independent microservice developed in Python, which integrates Artificial Intelligence (*Deep Learning*) models to automate the digitization of the catalog through the semantic removal of photographic backgrounds and Optical Character Recognition (*OCR*) of washing labels.

Based on this digitized inventory, the system implements advanced business logic composed of three main engines that transform the application into an automated personal assistant. First, a context-aware recommendation algorithm crosses the available clean clothes with real-time weather variables (using the *OpenWeatherMap* API) to generate outfits fully adapted to the climate. Second, an expert system evaluates the composition and typology of each garment to suggest its optimal spatial organization within the physical wardrobe. Finally, a planner automatically groups dirty clothes into safe laundry loads, simultaneously applying constraints regarding colorimetry, fabric fragility, and maximum temperatures. Ultimately, this synergy of modern technologies and heuristic rules manages to transcend the concept of a static inventory, comprehensively optimizing both the user's time and the preservation of their garments.

Agradecimientos

A mi familia, por creer en mí incluso cuando yo misma dudaba de este proyecto. Por apoyarme en cada paso de este proceso, durante las noches largas frente al ordenador, por escuchar mis frustraciones sobre cosas que probablemente no entendían, y por estar ahí siempre con una palabra de ánimo cuando las cosas se complicaban. Su confianza en que esto era posible me permitió seguir adelante en los momentos más difíciles.

Sin su apoyo constante, su paciencia infinita y su comprensión durante estos meses intensos de trabajo, este proyecto no habría sido posible. Este trabajo también es suyo.

Victoria López González

A mi familia, por enseñarme que el esfuerzo siempre vale la pena

Declaración de autoría

Yo, Victoria López González, declaro que soy la única persona autora del Trabajo Fin de Grado titulado *Aplicación móvil para la gestión digital del armario*, que no se infringen las leyes vigentes sobre propiedad intelectual, y que todo el material no original está apropiadamente atribuido a quienes corresponda.

Asimismo, declaro que, en caso de haber utilizado herramientas de Inteligencia Artificial durante la elaboración del trabajo, dicho uso ha sido responsable, transparente y acorde a la normativa de la universidad, sin comprometer la autoría ni la integridad intelectual.

Talavera de la Reina, 13 de julio de 2026

Fdo.:

Índice general

Resumen	9
Abstract	11
Agradecimientos	13
Índice general	19
Índice de cuadros	23
Índice de figuras	25
Listado de acrónimos	27
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Objetivos	2
1.3. Estructura del documento	3
2. Estado del Arte	5
2.1. El problema del vestuario personal y el consumo textil	5
2.1.1. La carga cognitiva de la decisión diaria	6
2.1.2. La infrautilización del armario	6
2.1.3. El modelo <i>fast fashion</i> y su impacto medioambiental	7
2.1.4. Posicionamiento de la propuesta	7
2.2. Aplicaciones de gestión de vestuario	7
2.2.1. Soluciones de inventario digital	8
2.2.2. Soluciones con recomendación y estilismo asistido	8
2.2.3. La dimensión ausente: mantenimiento y espacio físico	9

2.2.4.	Carencias identificadas y posicionamiento de la propuesta	9
2.3.	Sistemas de recomendación sensibles al contexto	10
2.3.1.	Paradigmas clásicos y su aplicabilidad al dominio	10
2.3.2.	Recomendación consciente del contexto	11
2.3.3.	Teoría del color aplicada a la combinación de prendas	11
2.4.	Visión por computadora aplicada al dominio textil	12
2.4.1.	Segmentación de imágenes y eliminación de fondos	12
2.4.2.	Reconocimiento óptico de caracteres en entornos no controlados . .	13
2.5.	Sistemas expertos basados en reglas	13
2.6.	Tecnologías y paradigmas de desarrollo	14
2.7.	Conclusión	15
3.	Metodología	17
3.1.	Metodología para la gestión del proyecto: Scrum	17
3.1.1.	Pilares empíricos y valores de la metodología Scrum	17
3.1.2.	Equipo Scrum	18
3.1.3.	Eventos de Scrum	18
3.1.4.	Artefactos Scrum	19
3.2.	OpenUp como metodología de desarrollo Software complementaria a Scrum	20
3.2.1.	Fases del ciclo de vida e integración con Scrum	20
3.2.2.	Integración de las metodologías en el proyecto	21
3.3.	Control de versiones en Git	24
3.3.1.	Estrategia de ramificación (<i>Branching Strategy</i>)	25
3.3.2.	Flujo de trabajo técnico	25
3.3.3.	Versionado del Proyecto	26
4.	Solución propuesta y herramientas	29
4.1.	Arquitectura del sistema	29
4.1.1.	Frontend	29
4.1.2.	Backend as Services (BaaS)	32
4.2.	Modelo de datos relacional	39
4.3.	Lógica de Negocio y Flujos Principales	41
4.3.1.	Flujo de digitalización y análisis de prendas	42
4.3.2.	Motor de generación de conjuntos basado en contexto	43

<i>ÍNDICE GENERAL</i>	21
4.3.3. Algoritmo de organización y reglas de almacenamiento	46
4.3.4. Planificador inteligente de coladas (optimización del lavado)	46
5. Resultados	51
5.1. Acceso y registro de usuarios	51
5.2. Digitalización de prendas	52
5.2.1. Proceso de digitalización	53
5.3. Ficha de prendas y gestión de estados	54
5.4. Edición y eliminación de prendas	55
5.5. Generación de conjuntos por contexto	56
5.6. Guardado y creación manual de conjuntos	57
5.7. Lavandería inteligente	58
5.8. Configuración de la lavadora y perfil de usuario	58
5.9. Organización del armario físico	59
6. Conclusiones	61
6.1. Grado de consecución de los objetivos	61
6.2. Aportaciones y competencias adquiridas	62
6.3. Limitaciones	64
6.4. Líneas de trabajo futuro	64
6.5. Conclusión final	65
A. Anexo A	69
A.1. Declaración de uso de IA	69
A.2. Repositorio del Proyecto	70
A.2.1. Contenido del Repositorio	70
A.2.2. Estructura de Ramas	70
A.2.3. Ejecución Local	70
Referencias	71

Índice de cuadros

2.1. Comparativa de <i>ArmarioVirtual</i> frente a aplicaciones del mercado	10
---	----

Índice de figuras

3.1. Marco de trabajo Scrum	20
3.2. Ciclo de vida de OpenUP	21
3.3. Milestones del proyecto en Github	23
3.4. Issues del proyecto	23
3.5. Tablero Kanban en Github	24
3.6. Historial de Ramas en Github	26
3.7. Commits y ramas	27
4.1. Arquitectura del sistema	30
4.2. Imagen del modelo relacional de Supabase	40
4.3. Flujo del sistema de recomendación sensible al contexto	45
5.1. Inicio de Sesión y Creación de Usuarios	52
5.2. Pantalla de registro de prenda	53
5.3. Proceso de subir una prenda	54
5.4. Gestión de las características de la prenda	55
5.5. Visualización pantalla prendas y eliminación de prendas	56
5.6. Generar un conjunto pseudoaleatoriamente	57
5.7. Creación manual de un conjunto	57
5.8. Funcionalidad del lavado de prendas	58
5.9. Configuración de la lavadora y perfil de usuario	59
5.10. Organización del armario y Cierre de sesión	60

Listado de acrónimos

TFG	Trabajo Fin de Grado
OCR	Optical Character Recognition
PWA	Aplicaciones Web Progresivas
HTML	Hypertext Markup Languaje
UI	User Interface
BAAS	Backend as a Service
ACID	Atomicidad, Consistencia, Aislamiento y Durabilidad
RLS	Row Level Security / Políticas de Seguridad a Nivel de Fila
IDOR	Vulnerabilidades de Acceso Directo a Objetos
MCC	Mobile Cloud Computing
AWS	Amazon Web Services
CNN	Redes Neuronales Convolucionales
CRNN	Red Neuronal Recurrente Convolucional
DIS	Segmentación Dicotómica de Imágenes
RGB	Red, Green, Blue
PNG	Gráficos de Red Portátiles
RGBA	Rojo, Verde, Azul, Alfa
STR	Scene Text Recognition/Texto en Escenas Naturales
CTC	Connectionist Temporal Classification
BI-LSTM	Memoria a Corto y Largo Plazo Bidireccionales
CARS	Context-Aware Recommender System
SOA	Arquitectura Orientada a Servicios
3NF	Tercera Forma Normal
JSON	JavaScript Object Notation
GPS	Global Positioning System
API	Application Programming Interface
SQL	Structured Query Language
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token

MVP	Minimum Viable Product
REST	Representational State Transfer
JSI	JavaScript Interface
PK	Primary Key
SQL	Structured Query Language

Capítulo 1

Introducción

ESTE documento constituye la memoria técnica y descriptiva del Trabajo de Fin de Grado (TFG) centrado en el diseño, desarrollo e implementación de una aplicación denominada *ArmarioVirtual*. Este proyecto nace con el propósito de aplicar los principios de la ingeniería del *software* y la integración de tecnologías de vanguardia para resolver una problemática cotidiana: la gestión ineficiente del vestuario personal. A lo largo de los capítulos que componen esta memoria se documentará de manera rigurosa el ciclo de vida completo del sistema, abarcando desde su concepción teórica y justificación metodológica, hasta su diseño arquitectónico y validación funcional. Como punto de partida, de este primer capítulo establece el marco conceptual del proyecto, detallando el contexto que motiva la propuesta, la definición exacta del problema a resolver y los objetivos estratégicos que han guiado el desarrollo de la solución.

1.1 Contexto y motivación

La gestión del vestuario personal es una actividad cotidiana que, pese a su aparente trivialidad, concentra una carga cognitiva y logística considerable. Estudios sobre los hábitos de consumo y organización doméstica señalan que las personas dedican una media de entre diez y dieciséis minutos diarios a decidir qué ropa ponerse [1], una decisión condicionada por variables meteorológicas, el contexto social del día, la disponibilidad de prendas limpias y las restricciones de almacenamiento físico que tenga el armario de cada individuo.

A esta problemática se suma el impacto del modelo de consumo textil actual. La industria de la moda es, según datos de la Agencia Europea del Medio Ambiente, uno de los sectores de mayor huella medioambiental del planeta [2], impulsado por el fenómeno del *fast fashion* por el desaprovechamiento sistemático del vestuario existente.

Investigaciones en el ámbito del comportamiento del consumidor estiman que entre el más de la mitad de las prendas que componen un armario medio no se utilizan con regularidad [1], en muchos casos por desconocimiento de las posibilidades combinatorias disponibles o por falta de un sistema que oriente al usuario en su uso.

Desde el punto de vista tecnológico, el ecosistema de aplicaciones móviles orientadas a

1. INTRODUCCIÓN

la gestión del vestuario ha experimentado un crecimiento notable en la última década. Sin embargo, una revisión del mercado actual revela que la mayoría de estas soluciones se limitan a una única funcionalidad de inventario digital, permitiendo fotografiar y categorizar prendas, pero sin ofrecer inteligencia aplicada sobre ese catálogo. Las capacidades de recomendación contextual, la planificación automática de coladas o la optimización del espacio físico del armario son funcionalidades prácticamente inexistentes en las aplicaciones de uso general disponibles.

Esta brecha entre la problemática real del usuario y las soluciones disponibles en el mercado constituye la motivación principal de este Trabajo Fin de Grado (TFG). ArmarioVirtual se concibe como un asistente personal de vestuario que trasciende el concepto de inventario estático, incorporando tres motores de inteligencia aplicada que dan respuesta a los retos identificados:

- **Recomendación sensible al contexto:** un algoritmo heurístico que cruza en tiempo real las condiciones meteorológicas obtenidas mediante geolocalización GPS con las prendas del usuario disponibles, generando combinaciones coherentes a nivel cromático, funcional y climático
- **Sistema experto de organización espacial:** un motor de reglas que evalúa la composición textil y tipología de cada prenda para recomendar su método de almacenamiento óptimo, maximizando la durabilidad de los tejidos y el aprovechamiento del espacio disponible.
- **Planificador inteligente de coladas:** un algoritmo de agrupación multidimensional que clasifica automáticamente las prendas sucias en coladas seguras, aplicando restricciones simultáneas de colorimetría, fragilidad textil y temperatura máxima de lavado, teniendo en cuenta las especificaciones del electrodoméstico registrado por el usuario.

La confluencia de estas funcionalidades en una única plataforma móvil, sostenida por tecnologías de aplicación semántica de imágenes y el reconocimiento óptico de caracteres, justifica tanto la relevancia técnica como la utilidad práctica para el sistema desarrollado.

1.2 Objetivos

El objetivo general del presente trabajo es el diseño, desarrollo e implementación de una aplicación móvil para la gestión virtual del armario que automatice la digitalización del inventario mediante visión artificial y asista al usuario tanto en la selección de conjuntos como en el mantenimiento y lavado de sus prendas. Para dar respuesta a este objetivo general se definen los siguientes objetivos específicos:

- **Sistema de digitalización automatizada:** implementar un módulo que permita añadir prendas al inventario mediante fotografía, integrado algoritmos de procesamiento de imagen basados en redes neuronales convolucionales para eliminar el fondo automáti-

camente y obtener una representación visual limpia de cada artículo.

- **Motor de recomendación de conjuntos contextual:** desarrollar un generador de conjuntos que cruce en tiempo real las condiciones meteorológicas del usuario con su inventario de prendas disponibles, aplicando reglas de compatibilidad cromática y restricciones climáticas.
- **Módulo de lavandería inteligente:** crear un planificador que utilice tecnología Optical Character Recognition (OCR) para extraer la composición textil y los requisitos de cuidado de las etiquetas físicas de las prendas, y un algoritmo de agrupación que clasifique automáticamente las prendas en coladas compatibles.
- **Motor de optimización espacial:** desarrollar un sistema experto que genere recomendaciones personalizadas de almacenamiento para cada prenda registrada, en función de su categoría, composición textil y nivel de formalidad.
- **Interfaz de usuario intuitiva y multiplataforma:** diseñar y desarrollar una experiencia de usuario accesible que integre todas las funcionalidades anteriores, incorporando un asistente de estilo que genera conjuntos adaptados al contexto y ofrece consejos personalizados de mantenimiento de tejidos por prenda.

Adicionalmente, se propuso como objetivo secundario la integración de un sistema de web scraping para analizar catálogos de tiendas en línea y sugerir nuevas adquisiciones compatibles con el armario del usuario. Esta funcionalidad quedó fuera del alcance final del proyecto y se identifica como línea de trabajo futuro en el capítulo de conclusiones

1.3 Estructura del documento

El resto de esta memoria se organiza en cinco capítulos. Cada uno aborda una fase concreta del ciclo de vida del proyecto, desde su fundamentación teórica hasta la validación de la solución construida. Se recomienda una lectura secuencial, si bien los capítulos 4 y 5 pueden consultarse de forma independiente por quien esté interesado únicamente en los aspectos técnicos y funcionales del sistema.

Capítulo 2: Estado del Arte

Revisión del estado del arte en aplicaciones de gestión de vestuario, sistemas de recomendación y tecnologías de procesamiento de imagen aplicadas al dominio textil.

Capítulo 3: Metodología

Descripción de las metodologías empleadas durante el desarrollo: Scrum como marco ágil y OpenUp como metodología de ingeniería del software complementaria.

Capítulo 4: Solución propuesta y herramientas

Descripción técnica del sistema: arquitectura distribuida, modelo de datos y detalle de los motores de lógica de negocio implementados.

1. INTRODUCCIÓN

Capítulo 5: Resultados

Presentación de los resultados obtenidos y validación funcional de los requisitos del sistema.

Capítulo 6: Conclusiones

Reflexión sobre los objetivos alcanzados, limitaciones encontradas y líneas de trabajo futuro.

Capítulo 2

Estado del Arte

EN este capítulo se expone un análisis en profundidad de los fundamentos teóricos y tecnológicos que sustentan el desarrollo del proyecto, así como el estudio de las soluciones existentes en el dominio sobre el que se asienta *ArmarioVirtual*. Se abordan las principales tendencias, soluciones digitales y tecnologías emergentes aplicadas a la gestión del vestuario personal, los sistemas de recomendación sensibles al contexto, la visión por computadora orientada al sector textil y los sistemas expertos basados en reglas, con el objetivo de delimitar los conceptos que resultan de especial relevancia para la comprensión del TFG. Comprender este ecosistema tecnológico y sectorial permitirá entender con mayor profundidad el valor añadido que aporta la solución propuesta y de qué manera responde a las carencias que presentan las aplicaciones de gestión de armario disponibles en la actualidad.

El capítulo se estructura siguiendo una lógica que parte de lo general para profundizar en lo particular. En primer lugar se caracteriza el problema desde una perspectiva sectorial y socioeconómica, situando la gestión del vestuario en el contexto del consumo textil contemporáneo y sus implicaciones medioambientales. A continuación se revisa el ecosistema de aplicaciones de gestión de armario existentes, identificando sus categorías y carencias. Después se desarrollan los tres pilares teóricos que articulan la inteligencia del sistema: los sistemas de recomendación sensibles al contexto, la visión por computadora aplicada al dominio textil y los sistemas expertos basados en reglas. Por último, se contextualizan las decisiones tecnológicas estructurales del proyecto. De este modo, cuando se finalice la lectura de este capítulo se habrá establecido un marco conceptual necesario para valorar críticamente las decisiones de diseño que se presentarán en los siguientes capítulos.

2.1 El problema del vestuario personal y el consumo textil

Antes de analizar las soluciones tecnológicas, conviene describir el problema desde una perspectiva sectorial, puesto que el valor de cualquier herramienta digital reside en la relevancia del problema que se aspira resolver. La gestión del vestuario constituye una actividad cotidiana cuya complejidad real ha sido objeto de estudio en los ámbitos del comportamiento del consumidor, la psicología de la decisión y la sostenibilidad medioambiental. Lo que en apa-

2. ESTADO DEL ARTE

riencia se presenta como una tarea trivial concreta, en realidad, supone una carga cognitiva y logística considerable que se repite a diario.

2.1.1 La carga cognitiva de la decisión diaria

La decisión sobre qué prendas ponerse cada día constituye un ejemplo paradigmático de lo que la psicología denomina fatiga de decisión [3]. Se trata de una decisión recurrente, condicionada por múltiples variables simultáneas como pueden ser qué prendas hay limpias o qué tiempo va a hacer en el momento de salir a la calle, que el individuo debe resolver en un tiempo reducido, y con frecuencia en un cierto estado de alerta. La acumulación de pequeñas decisiones de este tipo a lo largo del día contribuye al agotamiento de la capacidad deliberativa, fenómeno que ha llevado a algunas figuras públicas a adoptar deliberadamente un vestuario uniforme con el fin de eliminar esta carga. Como puede ser el caso de Mark Zuckerberg y su camiseta gris con vaqueros, Steve Jobs y su jersey de cuello alto negro o Carolina Herrera y su invariable camisa blanca [4].

Por lo tanto, la automatización o asistencia de esta decisión representa una forma de aliviar una carga cognitiva real y recurrente. Un sistema capaz de proponer combinaciones coherentes y contextualmente adecuadas libera al usuario de un proceso deliberativo repetitivo, aportando de esta forma, un valor tangible en su rutina diaria. Esta premisa constituye una de las motivaciones fundamentales del motor de recomendación de conjuntos que se desarrolla en este proyecto.

2.1.2 La infrautilización del armario

Más allá de la carga cognitiva, la literatura sobre hábitos de consumo textil coincide en señalar la existencia de un fenómeno generalizado de infrautilización del armario. Varios estudios sobre comportamiento del consumidor estiman que una fracción muy significativa de las prendas que componen un armario medio no se utiliza con regularidad [1]. Hay múltiples causas para que se dé este fenómeno y se refuerzan mutuamente: el desconocimiento de las posibilidades combinatorias que ofrecen las prendas ya poseídas, la escasa visibilidad de aquellas prendas relegadas al fondo del armario, la ausencia de un sistema que oriente al usuario sobre cómo y cuándo emplear cada artículo, y la tendencia psicológica a recurrir de forma reiterada a un subconjunto reducido de prendas «favoritas».

Esta infrautilización tiene una doble lectura. Desde la perspectiva del usuario, supone un desaprovechamiento económico. Desde la perspectiva sistémica, alimenta un ciclo de adquisición continua en el que el consumidor compra prendas nuevas no porque carezca de opciones, sino porque desconoce o no percibe las que ya posee. Romper este ciclo de forma que se haga visible y combinable el vestuario existente, constituye uno de los objetivos del sistema.

2.1.3 El modelo *fast fashion* y su impacto medioambiental

El fenómeno descrito se enmarca en un modelo de consumo conocido como *fast fashion* o moda rápida, caracterizado por ciclos de producción acelerados, una renovación constante de colecciones, precios reducidos y una rápida obsolescencia percibida de las prendas. Este modelo, consolidado durante las últimas décadas, ha transformado la relación del consumidor con la ropa, sustituyendo la lógica de la durabilidad por la de la novedad permanente.

Las consecuencias medioambientales de este paradigma han sido ampliamente documentadas por organismos internacionales. La industria de la moda figura de forma recurrente entre los sectores de mayor huella ecológica del planeta, con un impacto notable en el consumo de agua dulce, las emisiones de gases de efecto invernadero, el vertido de microplásticos y la generación de residuos textiles. La producción de una sola prenda puede implicar el consumo de miles de litros de agua, mientras que una proporción considerable de la ropa producida acaba en vertederos o incineradoras con escaso o nulo reaprovechamiento [2, 5].

Frente a este modelo lineal de «producir, usar y desechar», ha surgido el concepto de *economía circular* aplicada a la moda, que propone alargar el ciclo de vida de las prendas mediante estrategias de reutilización, reparación, reventa y, de manera fundamental, un mayor aprovechamiento de las prendas ya adquiridas. En este marco, las soluciones digitales orientadas a maximizar el uso del vestuario existente, a planificar su mantenimiento adecuado y a prolongar la vida útil de los tejidos adquieren una relevancia que trasciende la comodidad individual, alineándose con objetivos de sostenibilidad de alcance global [6].

2.1.4 Posicionamiento de la propuesta

ArmarioVirtual se posiciona precisamente en esta intersección entre la conveniencia personal y la sostenibilidad. A diferencia de las plataformas orientadas a la adquisición de nuevas prendas, el sistema se concibe como una herramienta destinada a optimizar el aprovechamiento, la combinación y la conservación del vestuario que el usuario ya posee. Esta orientación describe las tres funcionalidades principales del sistema: el motor de recomendación, que reduce la carga cognitiva de la decisión diaria y hace visibles combinaciones, que el usuario no había contemplado; el sistema experto de almacenamiento que prolonga la vida útil de los tejidos mediante una conservación adecuada; y el planificador de lavado, el cual, previene el deterioro prematuro de las prendas derivado de un lavado inadecuado. De este modo, el proyecto busca contribuir a contrarrestar las dinámicas de infrautilización y reemplazo temprano que caracterizan el consumo textil contemporáneo.

2.2 Aplicaciones de gestión de vestuario

El ecosistema de aplicaciones móviles orientadas a la digitalización del armario, habitualmente denominadas *closet apps* o *wardrobe apps*, ha experimentado un crecimiento sostenido en la última década, impulsado por la generalización de los teléfonos inteligentes y la

2. ESTADO DEL ARTE

mejora de sus capacidades fotográficas. Una revisión del estado actual del mercado permite agrupar estas soluciones en función del grado de inteligencia aplicada que incorporan sobre el catálogo de prendas del usuario, estableciendo una clasificación que va desde el mero registro pasivo hasta la asistencia activa.

2.2.1 Soluciones de inventario digital

La mayor parte de las aplicaciones de uso general se enmarcan en la categoría de inventario digital. Aplicaciones como *Stylebook*¹, una de las más veteranas y conocidas del sector, o *Smart Closet*² responden a este modelo: su propuesta de valor se centra en permitir al usuario fotografiar y catalogar manualmente sus prendas, organizándolas por categorías y atributos definidos por él mismo. Algunas incorporan además la composición manual de conjuntos arrastrando las prendas sobre un lienzo digital, un calendario de uso o estadísticas sobre las prendas más empleadas, como el coste por puesta (*cost per wear*) que populariza *Stylebook*.

Si bien estas soluciones resuelven el problema de la visualización del armario de manera que se ofrece al usuario una representación digital ordenada de su vestuario, presentan una limitación estructural de fondo: la inteligencia del sistema reside enteramente en el usuario. Resulta ilustrativo que, incluso en una aplicación tan consolidada como *Stylebook*, la función de creación automática de conjuntos se limite a combinar prendas de forma aleatoria, sin cruzar el contenido del armario con variables externas como el clima o la ocasión. La categorización es manual y, por tanto, propensa a inconsistencias y al abandono progresivo a medida que el catálogo crece.

La generación de conjuntos exige una intervención activa y deliberada, lo que, en términos de uso, provoca precisamente la carga cognitiva que el sistema debería aliviar. Y, además, no existe ninguna asistencia automatizada que cruce el contenido del armario con variables externas como el clima o la ocasión.

2.2.2 Soluciones con recomendación y estilismo asistido

Un segundo grupo incorpora capacidades de recomendación de estilo y, en ocasiones, integración con plataformas de comercio electrónico. Aplicaciones más recientes como *Whering*³ representan la vanguardia de esta categoría: además de digitalizar el armario, sugieren conjuntos e incorporan un discurso explícito de sostenibilidad y economía circular, promoviendo el aprovechamiento de las prendas ya poseídas frente a la compra impulsiva. Otras, como *Acloset*⁴, aplican técnicas de inteligencia artificial para proponer combinaciones e incluso para el recorte automático del fondo de las fotografías. En el extremo orientado al

¹www.stylebookapp.com

²smartcloset.me

³whering.co.uk

⁴www.acloset.app

comercio, servicios como Stitch Fix combinan estilistas profesionales con algoritmos para recomendar y enviar prendas nuevas al usuario.

No obstante, esta segunda generación presenta dos limitaciones recurrentes. En primer lugar, su lógica de recomendación tiende a orientarse hacia el descubrimiento y la compra de nuevo producto, más que hacia la optimización del vestuario existente, como ejemplifican los servicios de tipo *Stitch Fix*⁵. En segundo lugar, aun cuando algunas aplicaciones han empezado a incorporar el contexto, *Whering*, por ejemplo, integra ya la previsión meteorológica, esta integración suele ser superficial: no combina de forma rigurosa la disponibilidad real de las prendas, la compatibilidad cromática y las condiciones climáticas, ni contempla la planificación del mantenimiento de los tejidos. La recomendación opera así sobre un plano principalmente estético, desligada del contexto operativo que determina qué prendas están realmente disponibles y son adecuadas en cada momento.

2.2.3 La dimensión ausente: mantenimiento y espacio físico

Resulta destacable que prácticamente ninguna de las soluciones revisadas, incluidas las más avanzadas como *Whering*, aborde la dimensión física y logística del vestuario. La gestión de un armario real no se agota en saber qué prendas se posee el usuario y cómo combinarlas. Comprende también dónde y cómo almacenarlas para preservar su integridad, y cómo lavarlas de manera segura para prolongar su vida útil.

Estas dos dimensiones, la del almacenamiento óptimo y la del lavado seguro, constituyen problemas cotidianos que generan deterioro prematuro de las prendas y que cuando se resuelven de forma incorrecta provoquen que un jersey de punto se deforme o una prenda delicada lavada a temperatura excesiva encoja o destiña. Además, un armario mal organizado dificulta el acceso y favorece la infrautilización ya descrita.

La ausencia de estas funcionalidades en el mercado no es porque no sean relevantes, sino que resulta complicado automatizarlas, pues requieren codificar conocimiento experto sobre tejidos, métodos de conservación y restricciones de lavado. Es precisamente en este espacio donde *ArmarioVirtual* introduce una contribución diferencial respecto a las soluciones existentes.

2.2.4 Carencias identificadas y posicionamiento de la propuesta

Del análisis comparativo del mercado se desprende una brecha clara entre la problemática real del usuario y las funcionalidades ofrecidas por las soluciones disponibles. Las capacidades de recomendación verdaderamente sensible al contexto, la planificación automática del lavado y la optimización del espacio físico del armario son funcionalidades prácticamente inexistentes en las aplicaciones de uso general. El Cuadro 2.1 sintetiza este análisis, contrastando las capacidades habituales de las dos categorías de soluciones identificadas con el

⁵www.stitchfix.com

2. ESTADO DEL ARTE

alcance funcional propuesto para *ArmarioVirtual*.

Capacidad	Stylebook	Smart Closet	Whering	Acloset	Armario-Virtual
Digitalización del armario	Sí	Sí	Sí	Sí	Sí
Eliminación automática de fondo	Sí	Sí	Sí	Sí	Sí
Composición manual de conjuntos	Sí	Sí	Sí	Sí	Sí
Recomendación automática de conjuntos	Aleatoria	Aleatoria	Parcial	Parcial	Sí
Sensibilidad al clima en tiempo real	No	No	Parcial	Parcial	Sí
Compatibilidad cromática automática	No	Parcial	No	No	Sí
Planificación inteligente de lavado	No	No	No	No	Sí
Optimización del almacenamiento físico	No	No	No	No	Sí
Orientación a la sostenibilidad	Parcial	No	Sí	Parcial	Sí

Cuadro 2.1: Comparativa funcional entre *ArmarioVirtual* y las principales aplicaciones de gestión de vestuario, según las versiones analizadas durante la redacción de esta memoria

Como evidencia el cuadro, la propuesta no se limita a replicar funcionalidades existentes con una implementación alternativa, sino que aspira a cubrir dimensiones del problema que el mercado actual desatiende. Esta brecha constituye la justificación del enfoque adoptado, que integra en una única plataforma tres motores de inteligencia aplicada cuyo fundamento teórico se desarrolla en las secciones siguientes.

2.3 Sistemas de recomendación sensibles al contexto

La idea del generador de conjuntos de *ArmarioVirtual* se asienta sobre la teoría de los sistemas de recomendación, una disciplina consolidada dentro de la inteligencia artificial y la recuperación de información. Esta disciplina aborda el problema de seleccionar, dentro de un conjunto potencialmente amplio de elementos, aquellos más adecuados para un usuario y una situación concretos. Su relevancia ha crecido de forma paralela a la expansión del comercio electrónico y de las plataformas de contenido digital, donde la sobreabundancia de opciones hace inviable la exploración exhaustiva.

2.3.1 Paradigmas clásicos y su aplicabilidad al dominio

Tradicionalmente se distinguen tres grandes paradigmas: el *filtrado colaborativo* que formula recomendaciones a partir del comportamiento agregado de usuarios con preferencias afines; el *filtrado basado en contenido* que deriva las preferencias de los atributos descriptivos de los propios elementos y de un perfil de preferencias del usuario; y los *sistemas híbridos* que combinan ambas estrategias para mitigar sus debilidades, entre las que destaca el conocido problema del *arranque en frío* (*cold start*), el cual, penaliza a los sistemas colaborativos cuando carecen de un histórico de interacciones suficiente [7].

En el dominio específico de la recomendación de vestuario, el filtrado colaborativo resulta poco aplicable por una razón estructural: el armario de cada usuario constituye un conjunto privado y cerrado, sin solapamiento directo con el de otros usuarios. A diferencia de un catálogo compartido por toda la base de usuarios, en *ArmarioVirtual* cada usuario opera sobre su propio inventario exclusivo, por lo que recomendar un conjunto no consiste en sugerir un elemento de un catálogo global, sino en combinar prendas pertenecientes al conjunto privado del propio usuario.

Por ello, la implementación escogida se sitúa conceptualmente más próxima a la recomendación basada en contenido, enriquecida con un componente contextual que constituye su diferenciación.

2.3.2 Recomendación consciente del contexto

Los *Context-Aware Recommender System* (CARS) extienden el modelo clásico incorporando variables contextuales como una dimensión adicional en el proceso de selección. Mientras que un sistema tradicional opera sobre la dualidad usuario-elemento, un sistema CARS introduce una tercera dimensión, el contexto, que puede comprender la franja temporal, la ubicación geográfica, las condiciones ambientales o la situación social. La premisa subyacente es que la idoneidad de un elemento no es absoluta, sino que depende de la situación [8].

La idoneidad de un conjunto de ropa no depende únicamente de las preferencias estéticas del usuario, sino de un contexto que incluye de forma crítica las condiciones meteorológicas puesto que, por ejemplo, no tendría sentido recomendar un abrigo en un día de calor, y la ocasión de uso. La recomendación de vestuario es, por su propia naturaleza, un problema intrínsecamente contextual, lo que justifica la adopción del paradigma CARS como marco de referencia.

ArmarioVirtual lo materializa mediante un sistema basado en *heurística determinista*, en contraposición a los enfoques de aprendizaje automático que predominan en los sistemas de recomendación a gran escala. Esta elección responde a la naturaleza del dominio, cuyas reglas de compatibilidad climática y cromática pueden formalizarse de manera explícita, y a la ausencia de un corpus masivo de interacciones que justificara un modelo entrenado; sus implicaciones de diseño se desarrollan en el Capítulo 4.

2.3.3 Teoría del color aplicada a la combinación de prendas

Un componente esencial del motor de recomendación es la evaluación de la compatibilidad cromática entre las prendas que componen un conjunto. Este aspecto se fundamenta en la teoría del color, disciplina que estudia las relaciones de armonía y contraste entre tonalidades a partir de su disposición en el círculo cromático, y que proporciona un marco formal para determinar qué combinaciones resultan estéticamente coherentes.

2. ESTADO DEL ARTE

Los esquemas de armonía cromática clásicos ofrecen distintas estrategias de combinación: el esquema *monocromático* emplea variaciones de un mismo color; el de colores *análogos* combina tonalidades adyacentes en el círculo cromático; el *complementario* enfrenta colores opuestos, produciendo un contraste vivo; y el *triádico* distribuye tres colores equidistantes [9]. Existen, además, ciertos colores considerados neutros, como el negro, el blanco o el gris, que presentan una elevada versatilidad combinatoria al armonizar con la práctica totalidad del espectro. *ArmarioVirtual* traduce estos principios a una *matriz de compatibilidad cromática*, una estructura formal que codifica de manera discreta las relaciones de armonía entre las categorías de color del sistema [9], permitiendo evaluar una combinación mediante una operación de consulta determinista y eficiente. El detalle de esta estructura se aborda en la descripción de la solución.

2.4 Visión por computadora aplicada al dominio textil

La digitalización automatizada del inventario se apoya en técnicas de visión por computadora (*computer vision*) y aprendizaje profundo (*deep learning*), con el objetivo de reducir al mínimo la fricción del proceso de incorporación de prendas, sustituyendo la entrada manual de datos por la extracción automática de información a partir de fotografías.

En el núcleo de estas técnicas se encuentran las *Redes Neuronales Convolucionales* (CNN), una arquitectura especialmente adecuada para el procesamiento de datos con estructura espacial que aprende de forma jerárquica representaciones cada vez más abstractas de la imagen, sin necesidad de que un ingeniero diseñe manualmente los descriptores visuales [10, 11]. En esta sección se revisan las dos tareas implicadas: la segmentación de imágenes para el aislamiento de prendas y el reconocimiento óptico de caracteres sobre etiquetas textiles.

2.4.1 Segmentación de imágenes y eliminación de fondos

El aislamiento de la silueta de una prenda respecto de su fondo constituye un problema de *segmentación de imágenes*, entendida como la partición de una imagen en regiones semánticamente coherentes. La idea es estandarizar el catálogo visual del armario, presentando todas las prendas sobre un fondo uniforme y transparente con independencia de las condiciones en que fueron fotografiadas.

Los enfoques tradicionales basados en la detección de bordes o en leer el umbral según el color resultan insuficientes ante fondos complejos o escenarios habituales en una fotografía doméstica no controlada. Este reto se aborda mediante arquitecturas de tipo *codificador-decodificador* (*encoder-decoder*), en las que el codificador comprime progresivamente la imagen extrayendo características multiescala que capturan el contexto global, mientras que el decodificador reconstruye a partir de ellas una máscara de segmentación que preserva los detalles finos, como los bordes exactos de la prenda. Una variante particularmente relevante para este proyecto es la Segmentación Dicotómica de Imágenes (DIS), cuyo objetivo

específico es separar con alta precisión topológica un único objeto en primer plano de su entorno, generando una máscara binaria que se aplica sobre la imagen original como canal de opacidad [12].

La explotación de modelos preentrenados de esta familia, integrados a través de herramientas de código abierto, permite obtener representaciones visuales estandarizadas del vestuario sin necesidad de entrenar un modelo propio desde cero ni de recopilar un conjunto de datos etiquetado. Esta estrategia de reutilización resulta especialmente adecuada para un proyecto desarrollado por una única persona y con recursos computacionales acotados, al trasladar la complejidad del entrenamiento a modelos ya validados por la comunidad científica.

2.4.2 Reconocimiento óptico de caracteres en entornos no controlados

La extracción automática de la composición textil y de los requisitos de cuidado a partir de las etiquetas físicas plantea un problema de OCR. Sin embargo, no se trata de un escenario de OCR convencional. Mientras que el OCR clásico se aplica sobre documentos impresos con texto nítido y de alto contraste, las etiquetas textiles presentan condiciones más adversas que sitúan el problema en el ámbito del Scene Text Recognition/Texto en Escenas Naturales (STR) donde el texto puede aparecer deformado por los pliegues y la curvatura de la tela, la tinta se desgasta con los lavados sucesivos y las condiciones de iluminación son variables e impredecibles. Estos factores hacen fracasar a los motores de OCR basados en la comparación estática de patrones, diseñados para entornos controlados.

El problema del STR se resuelve mediante arquitecturas de Red Neuronal Recurrente Convolutiva (CRNN), que combinan una componente convolutiva, encargada de extraer los mapas de características visuales y tolerar las distorsiones del dominio, con una componente recurrente que modela la naturaleza secuencial del texto e interpreta los caracteres en su contexto. Las librerías de código abierto implementan este paradigma ofreciendo una robustez muy superior a la de los motores tradicionales [13]. La salida textual obtenida es procesada posteriormente mediante *expresiones regulares* para estructurar la información relevante en un formato apto para su persistencia y posterior explotación, tal y como se describe en el Capítulo 4.

2.5 Sistemas expertos basados en reglas

Tanto el motor de optimización del almacenamiento como el planificador de lavado de *ArmarioVirtual* asientan sus bases sobre el paradigma de los *sistemas expertos basados en reglas*, una de las ramas clásicas de la inteligencia artificial simbólica. Estos sistemas tienen como objetivo emular la capacidad de decisión de un experto humano en un dominio acotado, codificando su conocimiento de forma explícita mediante *reglas de producción* del tipo *condición-acción*. Su arquitectura clásica distingue tres componentes: una base de co-

2. ESTADO DEL ARTE

nocimiento, que contiene las reglas; una base de hechos, que recoge la información del caso concreto; y un motor de inferencia, que aplica las reglas sobre los hechos para derivar conclusiones.

La elección de este enfoque frente a uno de aprendizaje automático responde a las características del dominio del cuidado textil, y refuerza la coherencia metodológica con el motor de recomendación. Los sistemas basados en reglas ofrecen ventajas decisivas en dominios donde el conocimiento experto es explícito y estable: la trazabilidad de las decisiones donde cada conclusión puede explicarse identificando la regla que la ha producido, la ausencia de necesidad de datos de entrenamiento y el comportamiento determinista y reproducible. El cuidado de los tejidos constituye un dominio idóneo para este enfoque, regido por convenciones bien establecidas, recogidas de hecho en normativas internacionales de etiquetado textil, sobre la conservación adecuada de cada tipo de fibra.

En el proyecto, este paradigma sustenta dos funcionalidades. La primera es la optimización del almacenamiento físico, que a partir de los atributos de cada prenda determina su método de conservación óptimo buscando maximizar la durabilidad de los tejidos y el aprovechamiento del espacio de manera que se cubra una carencia que las soluciones del mercado desatienden. La segunda es la planificación inteligente del lavado. Es el caso de uso más complejo desde el punto de vista de la lógica de negocio. Se trata de un problema de agrupación de las prendas sucias en coladas seguras bajo restricciones múltiples y simultáneas.

Este planteamiento se relaciona con los problemas de *agrupamiento* o *clustering*, que se estudian en el aprendizaje no supervisado. Sin embargo, como en este proyecto las restricciones son explícitas y se obtienen directamente de etiquetas de cuidado, un enfoque basado en reglas resulta más apropiado y fácil de seguir que un algoritmo estadístico. La formalización de ambos motores, incluido el cálculo de la temperatura mínima de seguridad de cada colada se presenta en el Capítulo 4.

2.6 Tecnologías y paradigmas de desarrollo

Por último, el desarrollo móvil, los modelos de *Backend as a Service* y la arquitectura de microservicios constituyen los tres pilares sobre los que se asienta la implementación, y cada uno responde a un análisis de las alternativas disponibles en el estado actual de la ingeniería del software.

En cuanto al desarrollo móvil, el panorama actual ofrece tres grandes paradigmas. El desarrollo nativo puro que ofrece el máximo rendimiento y acceso al dispositivo, pero duplica el esfuerzo de desarrollo y mantenimiento para cada plataforma. Las PWA ofrecen ventajas de distribución y mantenimiento unificado, pero la literatura documenta sus limitaciones para acceder al *hardware* de bajo nivel, limitación crítica para una aplicación que depende de la captura fotográfica y de la geolocalización. Entre ambos extremos, los *frameworks* de

tipo *runtime*, como React Native, combinan el renderizado de componentes nativos con una elevada reutilización de código.

Respecto a la infraestructura del servidor, los modelos BAAS abstraen la complejidad de la infraestructura y ofrecen servicios de persistencia, autenticación y almacenamiento listos para su consumo, lo que permite concentrar el esfuerzo en la lógica de negocio. Este beneficio se enmarca en la tendencia más amplia de las arquitecturas *serverless*, cuyas limitaciones de memoria y de tiempo máximo de ejecución resultan especialmente relevantes para las tareas de inferencia de modelos de aprendizaje profundo.

Esta restricción justifica el recurso a una *arquitectura de microservicios*, que estructura la aplicación como un conjunto de servicios independientes y débilmente acoplados: en *ArmarioVirtual*, el aislamiento del procesamiento intensivo de imágenes en un microservicio dedicado permite delegar las tareas de inferencia de forma asíncrona, sin comprometer la fluidez de la interfaz ni verse limitado por las restricciones de las plataformas *serverless*. Estos fundamentos sustentan la arquitectura cliente-servidor distribuida adoptada, que se describe en detalle en el Capítulo 4.

2.7 Conclusión

A lo largo de este capítulo se ha construido el marco conceptual que sustenta el desarrollo de *ArmarioVirtual*, recorriendo un itinerario que parte del problema sectorial y converge en sus fundamentos técnicos. Se ha caracterizado el problema del aprovechamiento del vestuario, situándolo en la intersección entre la carga cognitiva de la decisión diaria, la infrautilización del armario y el impacto medioambiental del modelo *fast fashion*, lo que justifica la orientación del proyecto hacia la optimización del vestuario existente y la sostenibilidad. La revisión del ecosistema de aplicaciones de gestión de armario ha evidenciado una brecha consistente entre las necesidades reales del usuario y las funcionalidades disponibles, especialmente en las dimensiones del mantenimiento textil y la gestión del espacio físico.

Sobre esta base, se han desarrollado los tres pilares teóricos que articulan la inteligencia del sistema: los sistemas de recomendación sensibles al contexto, que sustentan el generador de conjuntos mediante un enfoque heurístico determinista y la teoría del color; la visión por computadora aplicada al dominio textil, que habilita la digitalización automatizada mediante segmentación dicotómica de imágenes y reconocimiento de texto en escenas naturales; y los sistemas expertos basados en reglas, que fundamentan los motores de almacenamiento y lavado. El análisis confirma que la propuesta no constituye una réplica de soluciones preexistentes, sino una integración original de técnicas consolidadas orientada a cubrir un espacio funcional desatendido. Con este marco establecido, los capítulos siguientes detallan la metodología de desarrollo empleada y la materialización técnica de la solución.

Capítulo 3

Metodología

EN esta sección, se presentan las metodologías utilizadas durante todo el desarrollo de la solución. Se presentan la metodología seguida para la correcta gestión del proyecto y la metodología implementada para el desarrollo *software* de la solución.

3.1 Metodología para la gestión del proyecto: Scrum

Con el objetivo de gestionar el proyecto correctamente se ha decidido optar por Scrum como metodología ágil de gestión de proyectos. Scrum es una metodología ligera diseñada para generar valor a través de soluciones adaptables a problemas complejos. Se ha seleccionado este marco por su enfoque iterativo e incremental, el cual permite avanzar de forma paulatina, aplicar experimentación continua y obtener retroalimentación constante para aprender y mejorar sobre la marcha [14]. Scrum es una metodología basada en el empirismo, afirmando que el conocimiento proviene de la experiencia y de la toma de decisiones fundamentadas en la observación. El uso de esta metodología proporciona la estructura necesaria para generar valor de forma colaborativa para optimizar la previsibilidad y controlar los riesgos asociados al proyecto

3.1.1 Pilares empíricos y valores de la metodología Scrum

El funcionamiento de Scrum como proceso empírico se sostiene sobre tres pilares fundamentales:

- **Transparencia:** el proceso y el trabajo emergentes deben ser visibles tanto para los que realizan el trabajo como para quien lo recibe. Los artefactos que tienen poca transparencia pueden conducir a decisiones que disminuyen el valor y aumentan el riesgo.
- **Inspección:** tanto el progreso como los artefactos de Scrum deben ser inspeccionados de manera frecuente y diligente para detectar varianzas o problemas potencialmente indeseables
- **Adaptación:** si algún aspecto del proyecto se desvía fuera de los límites aceptables o si el producto es inaceptable, debe realizarse un ajuste lo antes posible para minimizar la desviación adicional. Se espera que un equipo de Scrum se adapte en el momento en que aprenda algo nuevo por medio de la inspección.

3. METODOLOGÍA

Para que estos tres pilares funcionen y sea posible construir un entorno de confianza, la metodología exige que las personas interioricen y apliquen cinco valores clave: compromiso, enfoque, apertura, respeto y coraje.

3.1.2 Equipo Scrum

La unidad central de esta metodología es el equipo Scrum, un grupo cohesionado de, como máximo, diez profesionales que no presenta jerarquías ni subequipos y que está integrado por un *Scrum Master*, *Product Owner* y los Desarrolladores. Este equipo es estrictamente multifuncional y autogestionado, lo que significa que posee todas las habilidades necesarias para decidir de forma autónoma qué, cómo y cuándo se ejecuta el trabajo. Gracias a su tamaño reducido se maximiza la agilidad y la comunicación, si el grupo creciera en exceso, deberá convertirse en unidades más pequeñas que compartan el mismo objetivo y *Product Backlog*. La organización hace responsable a este equipo de todas las actividades del ciclo de vida del proyecto desde la experimentación inicial hasta el mantenimiento final. Al estructurar su trabajo mediante Sprints a un ritmo sostenible, el equipo completo asume el compromiso indivisible de entregar un incremento de valor verdaderamente útil en cada iteración.

- **Scrum Master:** el líder responsable de establecer correctamente la metodología Scrum y garantizar la máxima efectividad del Scrum Team. Su principal labor consiste en capacitar a los desarrolladores en autogestión, facilitar todos los eventos y eliminar cualquier obstáculo que frene el progreso continuo. Además, asiste activamente al *Product Owner* en la gestión del trabajo y guía a toda la organización en la adopción estructural de este marco ágil.
- **Product Owner:** es la persona responsable de maximizar el valor del proyecto y de gestionar eficazmente el valor *Product Backlog*. Aunque puede delegar tareas, mantiene la responsabilidad final de definir los objetivos, priorizar los elementos de trabajo y asegurar la total transparencia de la lista de tareas. Para garantizar el éxito toda la organización debe respetar sus decisiones, las cuales se hacen visibles a través del orden de las tareas y del incremento final.
- **Desarrolladores:** son los profesionales del equipo comprometidos a construir un incremento útil y funcional en cada Sprint. Aunque sus habilidades técnicas varíen según el proyecto, asumen la total responsabilidad de planificar el *Product Backlog* (trabajo), asegurar la calidad técnica cumplimiento con la *Definición de Hecho*, adaptar su plan de acción para alcanzar las metas establecidas y mantener una estricta responsabilidad mutua como grupo autogestionado.

3.1.3 Eventos de Scrum

Scrum utiliza eventos formales con límite de tiempo para crear regularidad y minimizar la necesidad de reuniones no definidas. El evento contenedor principal es el Sprint , un ciclo de

trabajo de máximo un mes de duración durante el cual se construyen los incrementos. Dentro de cada Sprint se suceden los demás eventos:

- **Sprint Planning:** es el evento colaborativo que da inicio a cada iteración, donde todo el equipo establece el trabajo a realizar. En esta sesión se definen tres aspectos clave: el valor estratégico del ciclo, es decir, el objetivo del Sprint, qué elementos del *Product Backlog* se van a completar y el plan técnico detallado sobre cómo los desarrolladores ejecutarán las tareas. El objetivo de esta reunión, que tiene como duración máxima ocho horas para un mes, es la creación del *Product Backlog*.
- **Daily Scum:** es una breve reunión de sincronización de máximo quince minutos, exclusiva para los desarrolladores, que se celebra cada día en el mismo lugar y horario. Su propósito fundamental es inspeccionar el progreso hacia el Objetivo del Sprint, identificar posibles obstáculos y adaptar el plan de acción técnico para las siguientes 24 horas, fomentando así la autogestión y la toma rápida de decisiones.
- **Sprint Review:** es una sesión de trabajo colaborativa, con una duración máxima de cuatro horas para Sprints de un mes, donde el Equipo Scrum y las partes interesadas inspeccionan los resultados obtenidos. Su propósito es evaluar el progreso conjunto, analizar los cambios en el entorno y ajustar el *Product Backlog* para definir los próximos pasos y determinar las futuras adaptaciones hacia el objetivo del producto.
- **Sprint Retrospective:** es el evento que concluye la iteración, con una duración máxima de tres horas para Sprints de un mes. Su propósito es que el equipo Scrum inspeccione su propio desempeño respecto a personas, interacciones, procesos y planifica mejoras inmediatas y accionables para aumentar la calidad u la eficiencia en los siguientes ciclos de trabajo.

3.1.4 Artefactos Scrum

Los artefactos representan el trabajo o el valor generado y están diseñados para maximizar la transparencia de la información clave.

- **Product Backlog (pila de producto):** es una lista ordenada y emergente con todo lo necesario para el producto, siendo la única fuente de trabajo. Su compromiso es el Objetivo del Producto, una meta estratégica a largo plazo que guía al equipo y que debe cumplirse antes de asumir la siguiente.
- **Sprint Backlog (pila de sprint):** es el plan en tiempo real creado por los desarrolladores, compuesto por tareas seleccionadas para la iteración actual. Su compromiso es el Objetivo del Sprint (*Sprint Goal*), un propósito único que aporta coherencia y guía el trabajo diario de forma flexible.
- **Incremento:** es un avance concreto y utilizable que se suma a los progresos anteriores, rigurosamente verificado para funcionar en conjunto. Su compromiso es la Definición

3. METODOLOGÍA

de Hecho, el estándar de calidad exacto que toda tarea debe cumplir para considerarse terminada.

La figura 3.1 resume de forma esquemática el flujo completo del marco Scrum, mostrando la relación entre los artefactos, los eventos y el equipo descritos anteriormente.

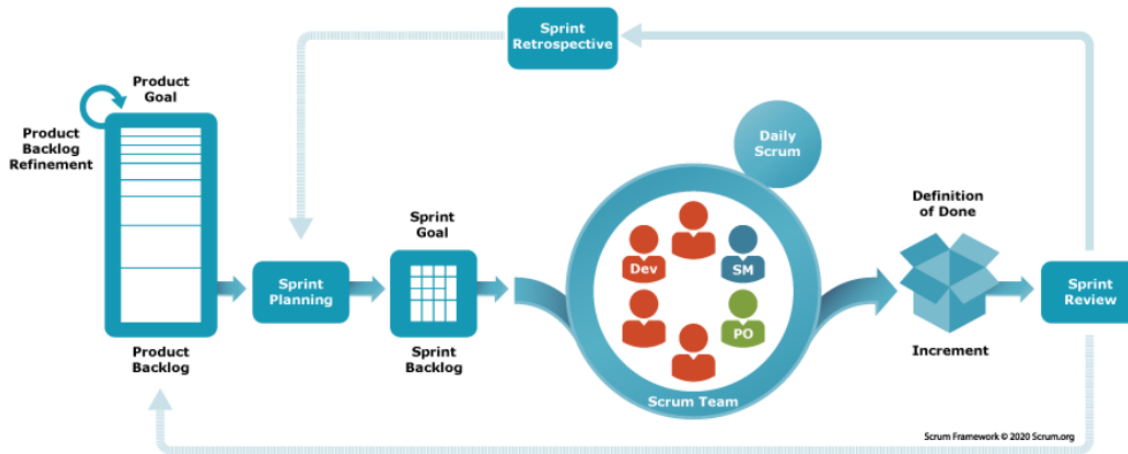


Figura 3.1: Esquema del marco de trabajo Scrum. Fuente: Scrum.org [14]

3.2 OpenUp como metodología de desarrollo Software complementaria a Scrum

Aunque Scrum proporciona un marco empírico excelente para la gestión del proyecto, la organización del tiempo y la adaptación continua, es una metodología agnóstica desde el punto de vista de ingeniería. Scrum carece de directrices de técnicas específicas sobre cómo capturar los requisitos del sistema, como diseñar la arquitectura base o cómo estructurar el ciclo de vida del código. Para cubrir esta dimensión técnica, se ha adoptado OpenUp como metodología complementaria para el desarrollo del software. La integración de ambas metodologías permite un enfoque completo: mientras que el progreso y la organización diaria se rigen por los Sprints y artefactos de Scrum, las decisiones de diseño, la mitigación de riesgos técnicos y la creación de los entregables se guían por los principios y fases de OpenUp.

3.2.1 Fases del ciclo de vida e integración con Scrum

Mientras que Scrum define la temporalidad del proyecto mediante Sprints de duración fija, OpenUp estructura el ciclo de vida del *software* basándose en la madurez técnica del producto. Para lograr una cohesión metodológica, las cuatro fases secuenciales de OpenUp se han mapeado y ejecutado a través de los Sprints de Scrum:

- **Fase de inicio:** el propósito es definir el alcance del proyecto y establecer su viabili-

dad. En esta etapa se identifican los requisitos principales, se definen los actores del sistema y se establece la visión global. El hito de esta fase asegura que todas las partes comprenden los objetivos y el alcance del sistema a desarrollar.

- **Fase de elaboración:** el objetivo es definir la línea base de la arquitectura y mitigar los riesgos técnicos de mayor impacto. el trabajo se centra en diseñar la estructura del sistema, seleccionar las tecnologías adecuadas y construir un esqueleto ejecutable. El hito de esta fase valida que la arquitectura es estable y que el proyecto es técnicamente viable.
- **Fase de Construcción:** comprende el desarrollo completo del sistema. Durante esta fase se diseñan, implementan y prueban el resto de los requisitos funcionales de forma iterativa, integrándolos progresivamente en la arquitectura previamente establecida. El hito que marca el final de esta etapa es la obtención de una versión operativa del *software* lista para ser desplegable en un entorno de pruebas.
- **Fase de Transición:** está orientada a la entrega del producto final. Las actividades principales incluyen la corrección de defectos encontrados en las fases de validación, la preparación del entorno de producción y la elaboración de la documentación definitiva. El hito finaliza con la aceptación formal y el lanzamiento del *software* [15, 16].

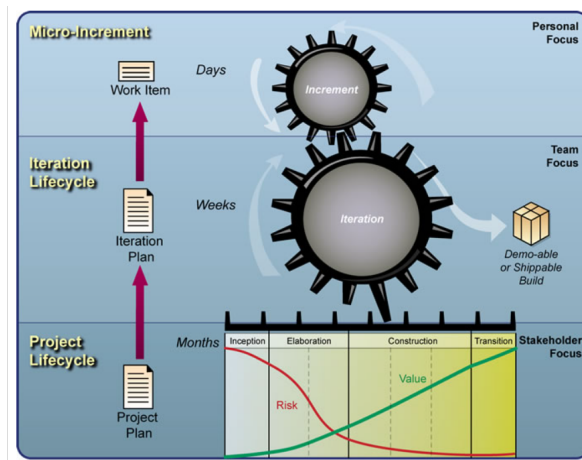


Figura 3.2: Niveles y ciclo de vida de OpenUP. Fuente: Eclipse Foundation [15]

La figura 3.2 ilustra la estructura de OpenUP en sus distintos niveles, así como la disposición secuencial de las cuatro fases y la evolución del riesgo y del valor a lo largo del ciclo de vida del proyecto

3.2.2 Integración de las metodologías en el proyecto

Tras establecer el marco teórico de ambas tecnologías resulta fundamental describir su aplicación práctica y su aplicación al contexto específico de este proyecto.

Dado que las metodologías anteriores, Scrum, orientada a la gestión del proyecto, y OpenUp, orientada a la ingeniería del software, están concebidas para equipos de trabajo con múltiples

3. METODOLOGÍA

integrantes, su implementación en un proyecto de carácter individual ha requerido un ajuste metodológico con el objetivo de mantener intactos los principios de agilidad, la calidad arquitectónica y la entrega continua de valor, pero eliminando la sobrecarga burocrática que no aporta beneficio a un entorno de desarrollo en solitario.

Para llevar a cabo esta integración, se ha seleccionado el ecosistema de Github, utilizando Github Projects para la gestión ágil y los repositorios de Github para el control de versiones. Además, la documentación técnica Github Projects permite centralizar la planificación mediante la creación de iteraciones (Sprints), la gestión del *Product Backlog* y el *Sprint Backlog* y el seguimiento visual del estado de las tareas a través de tableros *Kanban*¹ automatizados. En paralelo, el propio repositorio de Github aloja el código fuente y permite vincular cada avance directamente con las tareas planificadas, manteniendo la documentación técnica centralizada. Antes de detallar el uso específico de estas herramientas, resulta conveniente definir cómo se han materializado físicamente los roles teóricos de Scrum en el contexto de este TFG:

- **Product Owner:** este rol ha sido asumido por los tutores del proyecto Rubén Cantarero Navarro y Ana Rubio Ruiz, encargados de validar los requisitos, guiar el enfoque académico y priorizar el valor de las funcionalidades a desarrollar.
- **Scrum Master:** dada la naturaleza individual del proyecto, este rol de no ha requerido de ninguna figura independiente. Yo, en calidad de alumna, he asumido las labores de autogestión y control de la metodología.
- **Equipo de Desarrollo:** formado por una única integrante, la autora de este proyecto, Victoria López González, con la asunción de todas las responsabilidades técnicas dictadas por OpenUp.

Para organizar el volumen de trabajo, el proyecto se ha estructurado utilizando *Milestones* (Hitos) o agrupaciones de alto nivel dentro de Github, las cuales han hecho coincidir con las cuatro grandes fases del ciclo de vida de OpenUp. No obstante, dada la naturaleza individual del proyecto y con el fin de agilizar el proceso, las dos primeras etapas (inicio y elaboración) se han unificado en un único hito inicial. Esta decisión permite abordar de manera conjunta la definición de requisitos y el diseño de la arquitectura base. De esta manera, el trabajo queda dividido operativamente en tres bloques inicio-elaboración, construcción y transición. De esta manera, los grandes bloques del trabajo se corresponden de manera real de forma que:

- **Inicio-elaboración:** agrupadas bajo el hito denominado «*Minimum Viable Product* (MVP)»

¹Los tableros *Kanban* son una herramienta visual de gestión de proyectos que permite organizar el trabajo, controlar el flujo de tareas y mejorar la eficiencia

- **Construcción:** estructurada en el hito «Arquitectura de datos y Backend»
- **Transición:** definida mediante el hito «Revisión y entrega»

En la figura 3.3 se muestran estos tres hitos tal y como se han configurado junto con su fecha límite y el número de Issues cerrados asociados a cada uno.

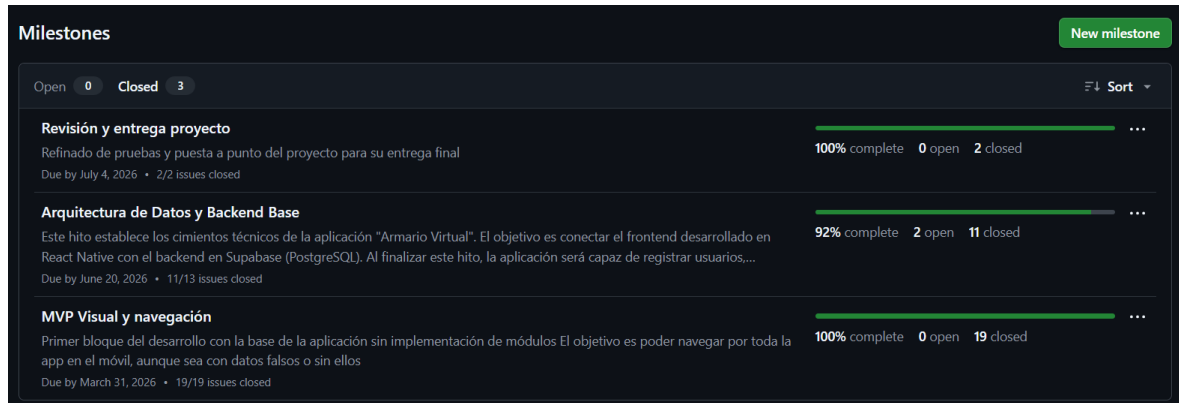


Figura 3.3: Milestones del proyecto en Github

Estos hitos generales se desglosan en tareas más pequeñas e individuales mediante la creación de *Issues* en Github. Cada *Issue* representa un requisito, un caso de uso o una técnica específica, conformando en su totalidad el *Product Backlog* del proyecto.

En la figura 3.4 se recoge una muestra de estos Issues, donde puede apreciarse su vinculación con el hito correspondiente y las etiquetas empleadas para clasificarlos según su ámbito.



Figura 3.4: Issues del proyecto

La dinámica del trabajo se ha gestionado mediante iteraciones o Sprints. Al inicio de cada Sprint se seleccionan los *Issues* pertinentes del *Product Backlog* y se trasladan al Sprint

3. METODOLOGÍA

Backlog. Para visualizar el flujo de trabajo diario, se ha configurado un tablero *Kanban* en Github Projects dónde cada tarea transita por estos cuatro estados principales:

- **Backlog (por hacer):** se corresponde al estado inicial de los Issues que han sido planificados para el Sprint actual, pero cuyo desarrollo aún no ha comenzado
- **In Progress (en progreso):** la tarea se encuentra actualmente en la fase de desarrollo o investigación.
- **In Review(en pruebas):** el código ha sido escrito, pero requiere validación, pruebas o revisión para asegurar que cumple con los requisitos esperados.
- **Done(completo):** la tarea ha sido finalizada con éxito, el código funciona correctamente y se ha integrado en el repositorio principal.

La figura 3.5 muestra el aspecto del tablero Kanban configurado en Github Projects, con las columnas correspondientes a cada uno de los estados anteriores y las tareas distribuidas según su situación en el flujo de trabajo.

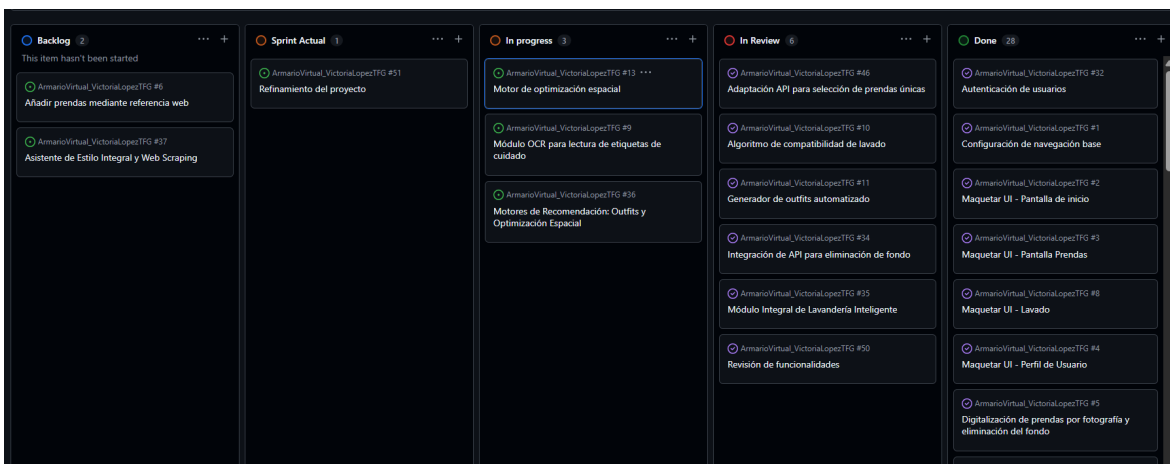


Figura 3.5: Tablero Kanban en Github

En cuanto a la temporalidad, la duración de los Sprints se ha fijado en dos semana. Esta cadencia se mantuvo constante a lo largo del desarrollo para establecer un ritmo de trabajo predecible. Si alguna de las tareas no lograba completarse dentro del marco temporal del Sprint, esta era devuelta al *Product Backlog* para ser revaluada y planificada en una iteración posterior. Por último para cubrir la necesidad de documentar el proceso continuo (simulando la inspección diaria del *Daily Scrum* y la gestión del conocimiento), se han empleado mensajes de commits de Git para trazar el progreso técnico diario.

3.3 Control de versiones en Git

Para dar continuidad a la explicación de herramientas el siguiente paso natural es detallar cómo se ha gestionado el código fuente. Con el objetivo de garantizar la integridad del có-

digo, mantener un historial detallado de cambios y facilitar el desarrollo iterativo propuesto por OpenUp, se ha implementado un sistema de control de versiones distribuido utilizando Git [17]. Como plataforma de alojamiento y gestión remota del repositorio, se ha optado por Github, lo que permite una integración directa con los tableros Kanban y la planificación del proyecto descrita en el apartado anterior.

El uso de un sistema de control de versiones resulta indispensable no solo como mecanismo de seguridad ante posibles pérdidas de información o errores de desarrollo, sino también como seguridad ante posibles pérdidas de información o errores de desarrollo, sino también como herramienta para cumplir con el pilar de la transparencia de Scrum. Cada incremento de software queda registrado, documentado y asociado a un objetivo específico del Sprint.

3.3.1 Estrategia de ramificación (*Branching Strategy*)

Dada la naturaleza individual del proyecto, no ha sido necesario implementar flujos de trabajo excesivamente complejos orientados a grandes equipos (como ocurre con un *GitFlow* adicional) . En su lugar, se ha optado por un flujo de trabajo simplificado basado en funcionalidades (*Feature Branch WorkFlow*), adaptado a una única desarrolladora. La estructura de ramas se ha organizado de la siguiente manera:

- **Rama principal (main):** contiene el código estable y funcional del proyecto. Esta rama refleja fielmente los incrementos de *software* finalizados al término de cada iteración y es la que se utiliza para evaluar la culminación de las diferentes fases de OpenUp también conocidas por hitos.
- **Ramas de funcionalidad (*feature branches*):** por cada nueva tarea o *Issue* seleccionada del *Sprint Backlog*, se crea una rama temporal e independiente a partir de la rama principal. Esto permite desarrollar y probar el nuevo código en un entorno aislado sin comprometer la estabilidad del sistema general.

Cabe destacar el uso de prefijos y guiones para todas las ramas creadas para modelar las funcionalidades como por ejemplo: *feature/captura-prendas* o *feature/eliminacion-fondo*

En la figura 3.6 se recoge el listado de ramas del repositorio, donde se observa la rama principal main, la rama develop y las distintas ramas de funcionalidad creadas siguiendo esta convención de nombrado.

3.3.2 Flujo de trabajo técnico

La implementación diaria del desarrollo ha seguido un ciclo de trabajo riguroso y estructurado:

1. Al iniciar el desarrollo de una tarea situada en la columna *Sprint Actual* del tablero Kanban, se genera su correspondiente rama de funcionalidad.
2. A medida que avanza el desarrollo, se realizan confirmaciones de código (*commits*)

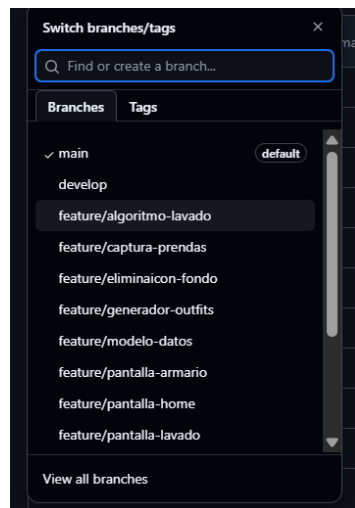


Figura 3.6: Historial de Ramas en Github

frecuentes. Se ha mantenido la política de mensajes de *commit* descriptivos y automáticos documentando claramente qué cambios se han introducido y por qué.

3. Una vez la funcionalidad cumple con los requisitos del Caso de Uso (OpenUp) y supera las pruebas locales, la tarea pasa a la fase de revisión.
4. Finalmente, la rama temporal, se fusiona (*merge*) con la rama principal mediante un *Pull Request* o fusión directa, y la tarea se marca como completada (Done) en Github Projects.

En la figura 3.7 se pueden observar una parte del esquema de ramas que aparece en el editor de Visual Studio Code, con los archivos modificados, las inserciones y todos los commits para describir los cambios en los archivos del proyecto. En concreto, aparece una pequeña visualización de la operación merge realizada a la rama main tras comprobar que el código implementado funciona de forma correcta.

3.3.3 Versionado del Proyecto

ArmarioVirtual presenta su versión 1.0.0, que marca la conclusión de este ciclo de desarrollo inicial completamente funcional. Esta versión integra todas las funcionalidades esenciales definidas en los objetivos del proyecto: el sistema de digitalización automatizada de prendas con OCR, el motor de recomendación de conjuntos sensible al contexto, el módulo de planificación inteligente de coladas y las sugerencias de optimización del espacio físico.

El desarrollo ha seguido una metodología ágil, con control de versiones mediante Git [18] y gestión visual del progreso a través de GitHub Projects, garantizando trazabilidad en cada etapa. Esta versión establece una base sólida y bien documentada sobre la que construir futuras mejoras identificadas durante el desarrollo, tales como el reconocimiento automático de símbolos de cuidado, la incorporación de aprendizaje personalizado a partir del comportamiento del usuario, y el despliegue en infraestructura escalable en la nube.

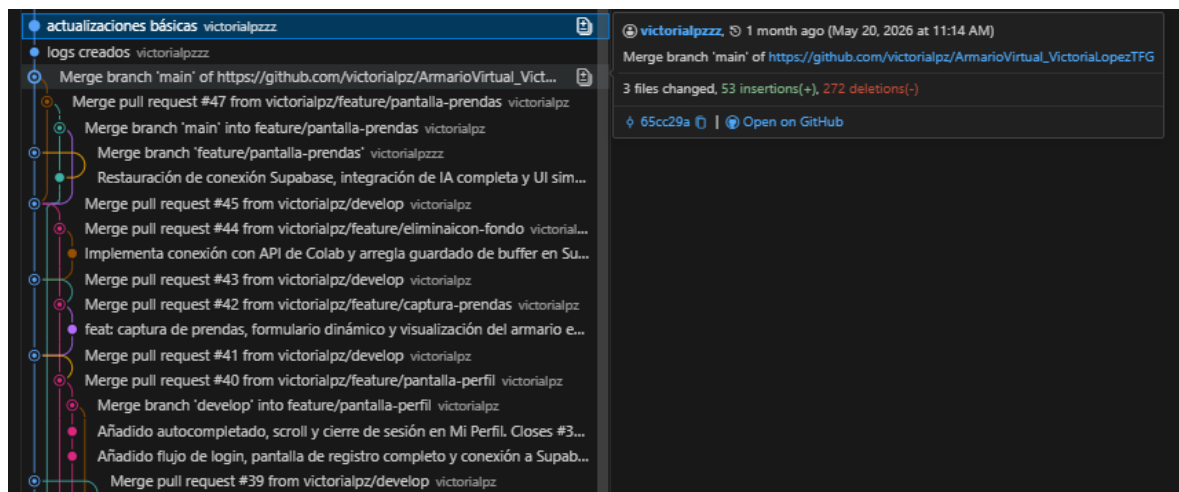


Figura 3.7: Commits y ramas

Capítulo 4

Solución propuesta y herramientas

TRAS haber establecido el marco metodológico que guía la planificación y el control del proyecto, este capítulo aborda el desarrollo técnico de la solución. El objetivo principal de este sistema es proporcionar una aplicación móvil integral que asista al usuario en la digitalización, organización física y gestión de su vestuario, apoyándose en la automatización de procesos como la lectura de etiquetas, la eliminación de fondos fotográficos y la generación de conjuntos basados en variables meteorológicas.

4.1 Arquitectura del sistema

La solución se ha diseñado siguiendo un modelo de arquitectura cliente-servidor distribuida, aplicando el principio fundamental de separación de responsabilidades. Este enfoque divide el sistema, la lógica de presentación, el almacenamiento persistente y el procesamiento intensivo.

Como se aprecia en la figura 4.1 esta arquitectura consta de de tres componentes principales: frontend (aplicación móvil), backend y servidor de procesamiento local

4.1.1 Frontend

Actúa como la interfaz de usuario principal y proporciona los mecanismos de captura de datos iniciados por el usuario. Sus responsabilidades abarcan la captura de imágenes de alta resolución, la obtención de coordenadas de geolocalización, la gestión del estado global de la interfaz y la comunicación bidireccional, mediante peticiones asíncronas HTTP/REST, con los servicios de backend en la nube y el microservicio local del Inteligencia Artificial.

Para la construcción de este componente, se ha evaluado el abanico actual de paradigmas de desarrollo móvil, decantándose finalmente por la tecnología multiplataforma React Native, soportada por el entorno Expo, frente a alternativas como *Aplicaciones Web Progresivas* (PWA) el desarrollo nativo puro. Decisión fundamentada en los siguientes criterios técnicos y arquitectónicos:

4. SOLUCIÓN PROPUESTA Y HERRAMIENTAS

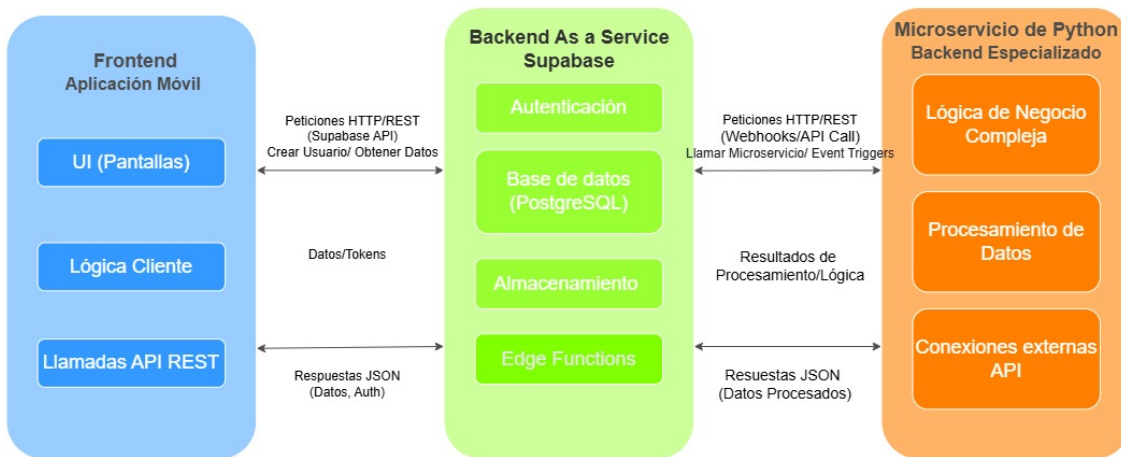


Figura 4.1: Arquitectura del sistema

Acceso a Hardware de bajo nivel

Se descartó el enfoque de PWA o aplicaciones híbridas basadas en WebView. Aunque las PWA ofrecen ventajas en términos de distribución web, estudios científicos y comparativas de rendimiento en desarrollo móvil señalan sus limitaciones para integrarse profundamente con el sistema operativo subyacente [19].

Dado que la precisión del sistema ArmarioVirtual depende críticamente de la captura de imágenes nítidas, para la lectura OCR de etiquetas y la extracción de fondos, y de la precisión del GPS para que el algoritmo genere conjuntos en base al color, al clima y el tipo de prenda, resulta imperativo tener acceso directo al *hardware*.

React Native, a diferencia de las aplicaciones híbridas tradicionales, no renderiza componentes HTML en un navegador web embebido, sino que invoca componentes UI verdaderamente nativos del sistema, como *UIButton* en iOS o *android.widget.Button* en Android [20]. Esto garantiza que las invocaciones a los sensores fotográficos y de localización se ejecuten a través de las APIs nativas del dispositivo, minimizando la latencia y maximizando la resolución y fiabilidad de los datos recolectados. En una PWA, el código se ejecuta dentro de la caja negra que conforma el navegador web, donde la comunicación entre el código *JavaScript* y la funcionalidad nativa subyacente se encuentra totalmente abstraída y bloqueada para el desarrollador.

Rendimiento computacional y arquitectura de hilos

Desde el punto de vista del rendimiento computacional, el diseño metodológico de React Native aporta una ventaja estructural fundamentada en su arquitectura de separación de hilos. En la ingeniería de aplicaciones móviles nativas, si una tarea de procesamiento intensivo, como

puede ser la serialización de una fotografía de alta resolución de una prenda para su posterior envío al microservicio de procesamiento, aparcara el hilo principal del sistema también conocido como *Main Thread* o *UI Thread*, se produce un bloqueo en la cola de procesamiento de eventos. La literatura técnica subraya que este bloqueo congela la interfaz gráfica, provocando la caída sistemática de fotogramas o *frame drops* deteriorando gravemente la experiencia y percepción de usuario [21].

Para mitigar este problema arquitectónico, React Native implementa un modelo de concurrencia que delega la ejecución de la lógica de negocio a un entorno de ejecución independiente, que generalmente es sostenido por motores como JavaScript o Hermes, el cual opera en un hilo secundario conocido como JavaScript Thread. Como explican Biørn-Hansen et al. (2020) en su estudio sobre las penalizaciones de rendimiento en aplicaciones multiplataforma [19], este enfoque, clasificado académicamente como *Runtime based*, permite combinar el renderizado de componentes de la interfaz de usuario de forma estrictamente nativa con un tiempo de ejecución aislado para la lógica algorítmica.

La comunicación entre el hilo lógico y los módulos nativos de renderización del sistema operativo no se produce de forma secuencial y bloqueante, sino de manera asíncrona. Tradicionalmente gestionada a través del JavaScript Bridge, las arquitecturas modernas del *framework* implementan interfaces avanzadas de comunicación directa como JavaScript Interface (JSI) y *TurboModules* que eliminan la necesidad de serializar los mensajes en formato JSON, permitiendo a JavaScript invocar funciones C++ de manera sincrónica y optimizada.

Esta separación arquitectónica resulta crítica para la viabilidad técnica de ArmarioVirtual. Garantiza que, mientras el dispositivo realiza operaciones asíncronas de entrada/salida, tales como comprimir la fotografía de una etiqueta o esperar el retorno de la clasificación meteorológica desde la API externa, el trabajo de interfaz no se interrumpa. Al aislar la carga computacional, las animaciones de la aplicación, la navegación mediante gestos y las transiciones entre pantallas se renderizan de forma autónoma en el hilo nativo, manteniendo una tasa estable de 60 fotogramas por segundo (FPS) y ofreciendo un flujo de uso fluido e ininterrumpido [19].

Viabilidad y eficiencia del ciclo de vida

En el contexto de un proyecto académico desarrollado por una única ingeniera, la gestión de recursos, como pueden ser el tiempo y el esfuerzo de mantenimiento, representa una variable restrictiva y fundamental. Los estudios recientes sobre la eficiencia en el desarrollo del software móvil avalan que los frameworks multiplataforma contemporáneos, y en particular React Native, destacan por su alta tasa de reutilización, permitiendo a los desarrolladores compartir hasta un noventa por ciento del código base en las plataformas iOS y Android [19].

4. SOLUCIÓN PROPUESTA Y HERRAMIENTAS

Adoptar un enfoque de desarrollo nativo puro requeriría la creación de repositorios independientes basados en lenguajes y herramientas específicas de cada plataforma como *Swift* para iOS y *Kotlin* para Android. Esta fragmentación metodológica no solo exige el dominio de múltiples ecosistemas técnicos, sino que conduce inevitablemente a una duplicación de la carga de trabajo, incrementando los costes, los tiempos de desarrollo y comprometiendo la viabilidad de la entrega en los plazos estipulados para desarrolladores individuales o equipos reducidos.

En este sentido, la adopción del ecosistema de React Native, apoyado por herramientas de enrutamiento y compilación como *Expo Router*, permite mitigar estos riesgos operativos de raíz. Esta combinación tecnológica no solo agiliza los ciclos de desarrollo y las fases de depuración mediante características nativas del framework como la recarga en caliente o *hot reloading* en inglés, sino que además logra conservar métricas de rendimiento y una experiencia de usuario competitiva frente a las soluciones nativas tradicionales [20].

4.1.2 Backend as Services (BaaS)

En la arquitectura de sistemas móviles contemporáneos, la creación y gestión de la infraestructura del lado del servidor, el *backend* suele representar un cuello de botella significativo hablando en términos de configuración, escalabilidad y mantenimiento de la seguridad. Para solventar este desafío y alinear la arquitectura con los requisitos de eficiencia y temporalidad del proyecto, se ha delegado la persistencia de los datos y la gestión de identidades a un modelo de computación en la nube conocido como *Backend as a Service*, implementado a través de la plataforma Supabase.

Las investigaciones sobre seguridad en la nube destacan que el modelo BaaS proporciona el soporte estructural esencial para las aplicaciones móviles para las aplicaciones móviles modernas, ya que simplifican radicalmente el desarrollo y la gestión centralizada de servicios críticos, tales como el almacenamiento masivo de datos, la autenticación de usuarios y las reglas de seguridad. En vez de realizar el diseño, el programa y desplegar un servidor relacional desde cero, el enfoque BaaS proporciona estos módulos listos para su consumo mediante interfaces de programación seguras.

Desde la perspectiva de la gestión del ciclo de vida del software, la adopción de arquitecturas tipo *serverless*, es decir, sin un servidor gestionado por el desarrollador, ayuda a construir aplicaciones a una velocidad mucho mayor. De manera que se elimina casi por completo la sobrecarga operativa de aprovisionar y gestionar infraestructura de *hardware* y *redes*. Al no existir dependencias complejas de despliegue, el esfuerzo técnico se destina exclusivamente a la programación de la lógica de negocio y la optimización de las consultas relacionales a la base de datos.

En el contexto específico de *ArtmarioVirtual*, la elección de Supabase permite integrar, bajo

un ecosistema centralizado la robustez de una base de datos relacional nativa, el almacenamiento de los ficheros de las prendas de vestir mediante *Storage Buckets* y el control de inicio de sesión seguro (Auth), cumpliendo con los estándares de la industria tecnológica sin comprometer los plazos de entrega [22].

Tecnologías de Backend y Almacenamiento

En la arquitectura de sistemas móviles contemporáneos, el diseño, despliegue y mantenimiento de la infraestructura del lado del servidor (backend) representa uno de los mayores desafíos técnicos, consumiendo una cantidad significativa de recursos en términos de configuración, escalabilidad y fortificación de seguridad. Para solventar este cuello de botella y alinear el proyecto con los principios del Desarrollo Rápido de Aplicaciones (RAD), se ha adoptado un paradigma de computación en la nube conocido como *Backend as a Service*. Supabase, una plataforma de código abierto que abstrae la complejidad de la infraestructura subyacente, ha sido la seleccionada para implementar esta capa. El uso de esta plataforma en *ArmaroVirtual* se fundamenta en tres pilares arquitectónicos esenciales: la reducción de carga operativa, el núcleo relacional basado en *PostgreSQL*, y la gestión integral de autenticación y almacenamiento de objetos.

1. **El paradigma BaaS y la reducción de sobrecarga operativa.** La literatura académica sobre computación en la nube destaca que el modelo BaaS proporciona un soporte estructural crítico para las aplicaciones modernas al eliminar la necesidad de administrar servidores físicos o máquinas virtuales. Tradicionalmente, el desarrollo de un *backend* a medida requiere la configuración manual de *frameworks*, el balanceo de carga, la gestión de conexiones a la base de datos y la aplicación de parches de seguridad. Supabase será quien adopte estas responsabilidades operativas delegadas al proveedor del servicio. Todo ello, permite que el esfuerzo de ingeniería se concentre casi en su totalidad en la lógica de negocio y en la experiencia de usuario del *Frontend*, maximizando la eficiencia en proyectos desarrollados por un único integrante [22].
2. **Núcleo relacional basado en PostgreSQL.** A diferencia de otras soluciones BaaS populares, Supabase está construido directamente sobre PostgreSQL, uno de los sistemas de gestión de bases de datos relacionales orientados a objetos más robustos y consolidados. Esta decisión arquitectónica es vital para la viabilidad de *ArmaroVirtual*. El dominio del problema exige un modelo de datos fuertemente estructurado y relacional: un usuario posee múltiples prendas (relación 1:N), y un conjunto de ropa está compuesto por múltiples prendas que, a su vez, pueden pertenecer a diferentes conjuntos (relación N:M). Optar por un esquema relacional con cumplimiento estricto de las propiedades Atomicidad, Consistencia, Aislamiento y Durabilidad (ACID), garantiza la integridad referencial de los datos, evitando anomalías en la actualización o borrado en cascada de los armarios de los usuarios [23].

3. **Gestión integral de Autenticación y Almacenamiento de objetos.** La plataforma centraliza dos requisitos funcionales críticos del sistema a través de APIs unificadas:

- **Supabase Auth:** es quien delega la complejidad criptográfica del inicio de sesión y el registro de usuarios. La plataforma gestiona la emisión, firma y validación de tokens *JSON Web Token (JWT)*. Un aspecto clave de esta integración es el uso de Row Level Security / Políticas de Seguridad a Nivel de Fila (RLS) nativas de PostgreSQL, lo que asegura a nivel de motor de base de datos, que un usuario autenticado únicamente pueda realizar consultas SELECT, INSERT, UPDATE, DELETE sobre sus propias prendas y conjuntos, previniendo las *Vulnerabilidades de Acceso Directo a Objetos (IDOR)* [24].
- **Supabase Storage:** proporciona un sistema de almacenamiento de objetos (*Object Storage*) optimizado para archivos multimedia pesados, como las fotografías de perfil y las imágenes procesadas de las prendas de vestir sin fondo. Esto separa eficientemente el almacenamiento de los metadatos, los cuales están ubicados en la base de datos relacional, de los archivos binarios, almacenados en buckets, optimizando el rendimiento general de lectura y escritura del sistema [23].

Para aplicar estos tres pilares en *ArmarioVirtual* hace falta configurar el acceso al *backend*. El algoritmo 1 muestra cómo se crea el cliente de Supabase a partir de la URL del proyecto y de la clave pública anónima. El control de acceso a los datos recae en las políticas RLS ya comentadas. La configuración de autenticación guarda la sesión en el almacenamiento local del dispositivo con *AsyncStorage*, de forma que el usuario no tenga que volver a iniciar sesión cada vez que abre la aplicación. Además, el token JWT se renueva solo antes de caducar mediante *autoRefreshToken*.

Como se trata de una aplicación móvil y no de una web, se desactiva *detectSessionInUrl*. El cliente se exporta una única vez para que toda la aplicación comparta la misma sesión.

Algorithm 1 Inicialización y configuración del cliente de Supabase

Require: *supabaseUrl* (URL del proyecto), *supabaseAnonKey* (Clave pública)

Ensure: *supabase* (Cliente instanciado y exportado)

- 1: *configAuth* $\leftarrow$ Crear diccionario de configuración
 - 2: *configAuth.storage* $\leftarrow$ *AsyncStorage* ▷ Habilita la persistencia local
 - 3: *configAuth.autoRefreshToken* $\leftarrow$ **true**
 - 4: *configAuth.persistSession* $\leftarrow$ **true**
 - 5: *configAuth.detectSessionInUrl* $\leftarrow$ **false**
 - 6: *supabase* $\leftarrow$ CREATECLIENT(*supabaseUrl*, *supabaseAnonKey*, *configAuth*)
 - 7: **Exportar** *supabase*
-

Este cliente es la única vía de acceso del *frontend* a los datos. Las tareas de procesamiento pesado quedan fuera de su alcance lo que motiva la descripción del microservicio descrito a continuación.

Microservicio de procesamiento

Con el objetivo de establecer el avance tecnológico del sistema propuesto, hay que destacar la infraestructura encargada del procesamiento intensivo de los datos. En la arquitectura de ArmarioVirtual módulos lógicos, como el motor de sugerencia de conjuntos operan mediante sistemas expertos basados en heurística determinista utilizando técnicas de Visión por Computadora o *Computer Vision* y de Aprendizaje Profundo o *Deep Learning*.

La decisión arquitectónica de aislar esta capa algorítmica en un microservicio de procesamiento independiente, desarrollado en Python empleando el framework web *FastApi*, responde a restricciones computacionales documentadas en la literatura de la ingeniería del software móvil.

Descarga computacional y limitaciones Serverless

Los modelos de redes neuronales pre-entrenados requieren la carga en memoria de cientos de megabytes de pesos y parámetros. La información sobre *Mobile Cloud Computing* (MCC) evidencia que ejecutar tareas de inferencia profunda localmente en el dispositivo incrementa drásticamente el tamaño del archivo instalable, provoca sobrecalentamiento térmico y agota rápidamente la batería del terminal, deteriorando la experiencia de usuario. Por otro lado, delegar estos algoritmos a funciones serverless comerciales, como pueden ser las Edge Functions de Supabase o AWS Lambda, resulta inviable, dado que este tipo de proveedores imponen límites estrictos de memoria dinámica y tiempos máximos de ejecución, los cuales podrían verse excedidos, por ejemplo, a la hora de procesar imágenes de alta resolución [25]. Por consiguiente, el despliegue de un microservicio dedicado se presenta como el estándar arquitectónico idóneo, permitiendo al Frontend realizar inferencias complejas de forma asíncrona y segura.

Las capacidades de procesamiento avanzado en este microservicio se materializan mediante dos flujos de procesamiento fundamentales:

Segmentación semántica y eliminación de fondos: El proceso de aislar la silueta de una prenda para estandarizar el catálogo visual del armario no se realiza mediante algoritmos tradicionales de procesamiento de imágenes, como podría ser la detección de bordes de *Canny* o umbrales de color, los cuales fallan ante fondos complejos o texturas similares. Este desafío se resuelve con un servicio que integra la herramienta *rembg*, el cual, emplea un modelo de CNN pre-entrenado denominado *isnet-general-use*, Image Segmentation Network.

Este modelo está diseñado específicamente para una tarea, conocida en la visión por computadora como DIS, cuyo objetivo es separar objetos de muy alta precisión topológica de su entorno. A nivel arquitectónico, la red neuronal opera mediante una estructura de codificador-decodificador (*encoder-decoder*) que extrae características jerárquicas y multiescala. Lo que permite al modelo comprender el contexto global de la imagen, identificando qué píxeles

4. SOLUCIÓN PROPUESTA Y HERRAMIENTAS

pertenecen estructuralmente a la prenda, sin perder los detalles finos locales como pueden ser los bordes exactos, pliegues o fibras del tejido.

A nivel de ingeniería, la inferencia de este modelo no borra el fondo directamente, sino que resuelve un problema de clasificación píxel a píxel generando una máscara binaria donde el valor 1 representa la prenda y 0 el fondo. Posteriormente, el microservicio procesa esta matriz bidimensional y la aplica mediante una operación a nivel de bits sobre la imagen original, inyectándola como el canal de opacidad *Alpha channel*. El resultado de este flujo es la transformación de una fotografía estándar RGB a un recurso gráfico estandarizado con fondo transparente (formato PNG RGBA), optimizado para la UI [12].

Reconocimiento óptico de caracteres en entorno no controlados: La digitalización de la información contenida en las etiquetas de lavado se ejecuta mediante la librería *EasyOCR*. A diferencia de los motores OCR clásicos basados en la comparación estática de matrices de píxeles, esta herramienta basa su funcionamiento en una arquitectura de CRNN que es el estándar actual para abordar el complejo problema del Reconocimiento de STR.

El flujo de procesamiento interno de este modelo se divide en tres etapas algorítmicas que operan en cadena:

1. **Extracción de características espaciales CNN:** En primer lugar, una red neuronal convolucional procesa la imagen de la etiqueta. Su función es extraer mapas de características visuales de alto nivel, lo que permite al modelo identificar patrones tipográficos y tolerar distorsiones severas muy comunes en el dominio textil, tales como letras deformadas por dobleces en la tela, arrugas, desgaste de la tinta o variaciones bruscas en la iluminación.
2. **Modelado de secuencias Bi-LSTM:** Los mapas de características extraídos se transforman en secuencias y alimentan a una red neuronal recurrente, empleando específicamente capas de Memoria a Corto y Largo Plazo Bidireccionales (Bi-LSTM). Esta arquitectura evalúa la probabilidad secuencial de las letras observando el contexto tanto hacia adelante como hacia atrás en vez de analizar los caracteres de forma aislada. Esto resulta crucial para predecir palabras con coherencia semántica.
3. **Decodificación y transcripción CTC:** Finalmente, la red utiliza una capa de Connectionist Temporal Classification (CTC). Este algoritmo probabilístico es clave, ya que permite alinear las secuencias temporales producidas por la capa recurrente con una cadena de texto final, identificando los caracteres válidos y eliminando duplicados sin necesidad de que las letras estuvieran perfectamente segmentadas en la imagen original.

A nivel de integración de software, la cadena de texto plano generada por la inferencia de *EasyOCR* es interceptada por la lógica de negocio del microservicio. Con la aplicación de técnicas de análisis léxico y Expresiones Regulares, *Regex*, el sistema extrae entidades de

información específicas, como el porcentaje numérico de los materiales, los tipos de tela y las temperaturas de lavado en grados Celsius, normalizando y estructurando esta salida en un formato JSON tipado, dejándolo completamente listo para su inserción transaccional en la base de datos relacional [13].

Integración de servicios a terceros y conciencia de Contexto

Para maximizar la utilidad y precisión de la aplicación, el diseño de ArmarioVirtual se convierte en un CARS. La efectividad de los motores de sugerencia en aplicaciones móviles aumenta significativamente cuando logran capturar e integrar variables del entorno del usuario en tiempo real, adaptando de forma dinámica sus salidas a la realidad de la situación.

Para implementar la sensibilidad meteorológica, el sistema adopta una Arquitectura Orientada a Servicios SOA mediante la integración de la API externa de *OpenWeatherMap*. El flujo de captura de contexto opera de la siguiente manera: En primer lugar, el *frontend*, o sea, la aplicación móvil, interactúa con los sensores de hardware del dispositivo para extraer las coordenadas GPS exactas, en concreto, la latitud y la longitud, del usuario en el momento de la consulta. Como mecanismo de respaldo, tal y como se recoge en el Algoritmo 2, el sistema también permite la introducción manual de la temperatura para poder planear conjuntos con vistas al futuro.

Algorithm 2 Clasificación climática manual

Require: *temp* (Temperatura introducida manualmente), *localidad* (Opcional, por defecto vacío)

Ensure: Objeto *clima* estructurado

```

1: function CALCULARCLIMADESDETEMP(temp, localidad)
2:   if temp < 15 then
3:     tipo ← 'Frío'
4:     descripcion ← 'frío'
5:   else if temp > 25 then
6:     tipo ← 'Calor'
7:     descripcion ← 'caluroso'
8:   else
9:     tipo ← 'Entretiempo'
10:    descripcion ← 'templado'
11:  end if
12:  clima ← Crear objeto con (temp, descripcion, tipo, localidad)
13:  return clima
14: end function

```

Después, estas coordenadas se envían mediante una petición asíncrona segura a los servidores de *OpenWeatherMap*, actuando este como un oráculo de datos ambientales.

Por último, el sistema recibe una respuesta estructurada en formato JSON con las condiciones meteorológicas actuales, y esta se filtra, normaliza y clasifica según la temperatura por

4. SOLUCIÓN PROPUESTA Y HERRAMIENTAS

la lógica de negocio de la aplicación, tal y como se detalla en el Algoritmo 3.

Algorithm 3 Clasificación climática basada en temperatura

Require: *data* (Respuesta JSON de la API de OpenWeatherMap)

Ensure: *clima* (Objeto estructurado con la temperatura y clasificación)

```
1: temp ← REDONDEAR(data.main.temp)
2: tipoClima ← 'Entretiempo'
3: if temp < 15 then
4:   tipoClima ← 'Frío'
5: end if
6: if temp > 25 then
7:   tipoClima ← 'Calor'
8: end if
9: clima ← Crear objeto
10: clima.temp ← temp
11: clima.descripcion ← data.weather[0].description
12: clima.tipo ← tipoClima
13: clima.localidad ← data.name
```

Lejos de realizar agrupaciones aleatorias en base a la temperatura, el algoritmo construye el conjunto evaluando simultáneamente cuatro dimensiones lógicas:

- **Lógica de Capas por Temperatura:** la temperatura obtenida se clasifica en tres rangos de acción estricta. Si el sistema detecta Frío, es decir, menos de quince grados centígrados, fuerza la búsqueda y adición de una prenda de la categoría Abrigo; si detecta Entretiempo, entre quince y veinticinco grados, restringe la capa exterior exclusivamente a prendas ligeras, como jerseys o sudaderas y, por último, en escenarios de Calor, más de veinticinco grados centígrados, se omite la capa exterior.
- **Motor de Reglas de Colorimetría:** Para garantizar que las combinaciones generadas sean coherentes, el sistema implementa una matriz de compatibilidad cromática, recogida en el Algoritmo 4. El algoritmo separa los colores neutros universales: blanco, negro, gris y beige que son quienes actúan como comodines de los colores de acento. A medida que se añade una nueva prenda al conjunto, el sistema valida que su color exista en el diccionario de compatibilidad del resto de prendas seleccionadas, descartando del espacio de búsqueda aquellas que generen disonancia visual.
- **Disponibilidad y Estado Operativo:** El algoritmo de recomendación no asume que todo el armario está disponible. Antes de la generación, consulta si el usuario desea aprovechar prendas que ya están En uso o A medias, para maximizar su ciclo de vida antes del lavado, o si la búsqueda debe restringirse estrictamente al subconjunto de ropa con estado Limpio.
- **Contexto de Evento y Estilo:** finalmente, la selección de prendas se categoriza y etiqueta en base al propósito social del usuario (Diario, Deporte, Formal, Fiesta, etc.),

permitiendo que el conjunto guardado en la base de datos mantenga una correlación directa con la ocasión de uso para la que fue creado.

Algorithm 4 Matriz y validación de compatibilidad cromática

Require: pA, pB (Prendas a evaluar, con atributo *color* como cadena separada por comas)

Ensure: **true** si los colores son compatibles, **false** en caso de disonancia

```

1: Definir conjunto NEUTRALES  $\leftarrow \{ \text{'Blanco'}, \text{'Negro'}, \text{'Gris'}, \text{'Beige'},$ 
2:    $\text{'Marrón'}, \text{'Dorado'}, \text{'Plateado'} \}$ 
3: Definir diccionario ACENTO_COMPAT  $\leftarrow$ 
    $\text{'Azul'} \rightarrow [\text{'Morado'}, \text{'Rosa'}, \text{'Naranja'}]$ 
    $\text{'Rojo'} \rightarrow [\text{'Rosa'}, \text{'Naranja'}]$ 
    $\text{'Verde'} \rightarrow [\text{'Amarillo'}, \text{'Azul'}]$ 
    $\text{'Amarillo'} \rightarrow [\text{'Verde'}, \text{'Naranja'}]$ 
    $\text{'Naranja'} \rightarrow [\text{'Rojo'}, \text{'Amarillo'}, \text{'Verde'}, \text{'Azul'}]$ 
    $\text{'Rosa'} \rightarrow [\text{'Morado'}, \text{'Rojo'}, \text{'Azul'}]$ 
    $\text{'Morado'} \rightarrow [\text{'Rosa'}, \text{'Azul'}]$ 
4: function COLORCOMPATIBLE( $pA, pB$ )
5:    $cA \leftarrow \text{SEPARARYLIMPIAR}(pA.\text{color}, ',')$  ▷ Lista de colores de la prenda A
6:    $cB \leftarrow \text{SEPARARYLIMPIAR}(pB.\text{color}, ',')$  ▷ Lista de colores de la prenda B
   ▷ Regla 1: Los colores neutros combinan con cualquier color
7:   if  $(\exists c \in cA \text{ tal que } c \in \text{NEUTRALES})$ 
8:   or  $(\exists c \in cB \text{ tal que } c \in \text{NEUTRALES})$  then
9:     return true
10:  end if
   ▷ Regla 2: Evaluación de compatibilidad de acentos
11:  for all  $a \in cA$  do
12:    for all  $b \in cB$  do
13:      if  $a = b$  or  $b \in \text{ACENTO\_COMPAT}[a]$  then
14:        return true
15:      end if
16:    end for
17:  end for
18:  return false
19: end function

```

De este modo, se garantiza que las sugerencias resultantes resuelven el problema térmico, están adaptadas a la estética visual, al ciclo de colada y a las condiciones climáticas del momento, proporcionando un valor añadido tangible que una aplicación de gestión de inventario tradicional no podría ofrecer.

4.2 Modelo de datos relacional

Para dotar al sistema de persistencia, garantizar la consistencia de la información y soportar las consultas complejas requeridas por los algoritmos de recomendación, se ha diseñado un modelo de datos fuertemente estructurado. Dicho modelo se encuentra implementado sobre el motor relacional PostgreSQL, que está gestionado a través del proveedor Supabase

4. SOLUCIÓN PROPUESTA Y HERRAMIENTAS

y ha sido normalizado hasta la Tercera Forma Normal (3NF) para eliminar redundancias y dependencias [23].

El esquema de la base de datos (ver Figura 4.2) se articula en torno a nueve tablas, divididas lógicamente según su dominio funcional. A continuación se describen las 6 principales:

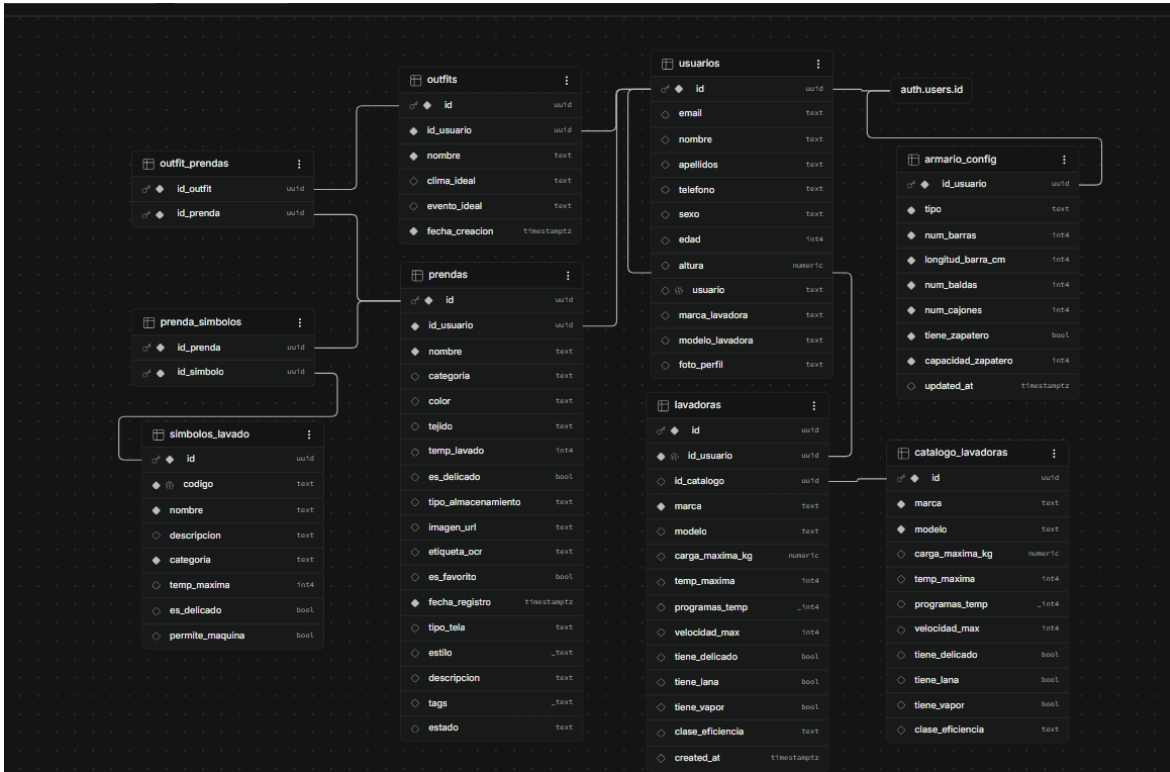


Figura 4.2: Imagen del modelo relacional de Supabase

- **Tabla usuarios:** la entidad raíz del modelo. Almacena la información demográfica y de perfil del usuario: e-mail, teléfono, nombre, apellidos, edad, altura, sexo, foto_perfil, etc. Su identificador principal (id) actúa como Primary Key (PK) y se encuentra estrictamente vinculado mediante un disparador (*trigger*) a la tabla de identidades internas de *Supabase Auth*, garantizando que todo registro de usuario provenga de una autenticación válida.
- **Tabla armario_config:** modela las propiedades físicas y la capacidad de almacenamiento del mobiliario del usuario: tipo, num_barras, num_baldas, capacidad_zapatero, etc. Mantiene una cardinalidad 1:1 con la tabla usuarios donde la clave primaria es simultáneamente clave foránea referenciando a id_usuario, debido a que en el alcance actual del sistema un usuario posee un único armario principal a configurar.
- **Tabla prendas:** representa el núcleo transaccional del sistema, almacenando el catálogo de ropa digitalizado. Mantiene una cardinalidad 1:N con el usuario, ya que un usuario puede registrar múltiples prendas. Incluye campos fuertemente tipados para

los metadatos extraídos por el microservicio de python como: `tipo_tela`, `temp_lavado` o `es_delicado` y el estado actual de la prenda: limpio, en uso, sucio y a medias. Para dotar al sistema de flexibilidad sin comprometer el esquema rígido de SQL, se ha implementado un campo *tags* utilizando el tipo de array de texto nativo de PostgreSQL. Esto permite indexar y consultar etiquetas personalizadas o atributos variables de las prendas de forma eficiente, combinando la robustez relacional con la flexibilidad de las bases de datos documentales.

La agrupación de prendas en conjuntos basados en la meteorología introduce una complejidad relacional clásica: un conjunto está formado por múltiples prendas, pero una misma prenda puede pertenecer a varios conjuntos diferentes. Para resolver esta cardinalidad de Muchos a Muchos (N:M), el diseño emplea una tabla intermedia o de ruptura:

- **Tabla outfits:** registra las agrupaciones lógicas creadas por el sistema o el usuario, almacenando metadatos de alto nivel como el `clima_ideal`, el `evento_ideal` y la `fecha_creacion`.
- **Tabla outfit_prendas:** actúa como tabla de resolución con una relación M:N donde clave primaria se compone por las claves foráneas `id_outfit` e `id_prenda`. Esta estructura garantiza la integridad referencial y permite ejecutar consultas de cruce eficientes (sentencias JOIN) para recuperar la composición exacta de cada conjunto.
- **Tabla catalogo_lavadoras:** es una entidad de soporte y solo lectura que almacena un catálogo estandarizado de electrodomésticos como son la marca, el modelo, la `carga_maxima_kg` y los programas. Esto permite a los algoritmos de la aplicación conocer las restricciones físicas de la lavadora del usuario a la hora de agrupar de forma inteligente sus prendas sucias.

Integridad y Seguridad a nivel de datos

Todo el modelo relacional implementa restricciones de borrado en cascada (ON DELETE CASCADE). De este modo, si se da de baja un usuario en el sistema, el motor de base de datos elimina automáticamente sus prendas, la configuración de su armario o sus conjuntos, se evitaría la persistencia de registros huérfanos y garantizando el cumplimiento de las normativas de protección de datos. Además, todas las tablas están protegidas por Políticas de Seguridad a Nivel de Fila (RLS), asegurando que las consultas SQL únicamente retornen las filas donde el `id_usuario` coincida con el identificador del token de sesión activo [26].

4.3 Lógica de Negocio y Flujos Principales

En este apartado se detalla la implementación de la lógica de negocio y se describen los flujos de ejecución principales que orquestan la interacción entre la UI, los servicios de

4. SOLUCIÓN PROPUESTA Y HERRAMIENTAS

procesamiento externo y la persistencia en la nube, garantizando el cumplimiento de los requisitos funcionales del sistema.

4.3.1 Flujo de digitalización y análisis de prendas

El proceso de añadir una nueva prenda al inventario representa uno de los flujos transaccionales más complejos de la aplicación, ya que requiere la coordinación asíncrona de múltiples componentes arquitectónicos. Para garantizar una experiencia de usuario fluida y evitar el bloqueo del hilo principal de la interfaz (*Main Thread*), la digitalización se ha estructurado como un proceso segmentado o *pipeline* de procesamiento secuencial gestionado mediante *Hooks* personalizados en React Native, apoyándose en los patrones de asincronía *async/await*, lo que facilita un modelo de concurrencia para coordinar las peticiones externas y el procesamiento de imágenes de forma no bloqueante.

El flujo operativo se inicia con la captura y selección de medios, en este momento la aplicación invoca las interfaces nativas del dispositivo para acceder a la cámara o a la galería, gestionando de forma transparente la solicitud y validación de permisos de hardware a nivel de sistema operativo. Durante esta etapa, se adquieren hasta dos imágenes: una fotografía frontal de la prenda y, de forma opcional, una captura detallada de su etiqueta de composición.

Una vez obtenidas, ambas imágenes se serializan y se transmiten mediante peticiones asíncronas HTTP/POST al microservicio dedicado, desarrollado en *FastAPI*. En este entorno, el procesamiento algorítmico se bifurca: por un lado, se ejecuta la herramienta *rembg* sobre la fotografía principal para aislar el objeto mediante redes neuronales convolucionales, de manera que se genera una imagen optimizada con fondo transparente en formato PNG; y paralelamente, si se proporcionó la imagen de la etiqueta, el motor *EasyOCR* extrae el texto plano empleando expresiones regulares para identificar la composición textil y las restricciones térmicas de lavado, devolviendo la salida en un formato estructurado JSON.

Tras esta fase de procesamiento profundo, el *frontend* recupera el archivo binario sin fondo y lo transfiere al *bucket* de almacenamiento de Supabase, obteniendo a cambio una URL pública como localizador del recurso.

Finalmente, el sistema orquesta la persistencia relacional agrupando esta URL, los metadatos extraídos por el OCR y los atributos introducidos manualmente por el usuario para ejecutar una inserción transaccional en la tabla de prendas. Este último paso se encuentra protegido por RLS, asegurando que el registro quede inequívocamente asociado al identificador de la sesión activa y finalizando el proceso con la actualización de la interfaz gráfica y del catálogo personal del usuario.

4.3.2 Motor de generación de conjuntos basado en contexto

Esta funcionalidad representa la transición del sistema de un simple inventario digital a un Sistema de Recomendación Sensible al Contexto. El objetivo de este flujo busca automatizar la toma de decisiones del usuario al vestirse, cruzando los datos de su armario con variables ambientales, operativas y estéticas en tiempo real [8]. El proceso, gestionado a través de un *Hook* dedicado `useGeneradorOutfits.ts`, se ejecuta mediante un algoritmo basado en reglas heurísticas estructurado en varias fases: primero se captura el contexto ambiental, es decir, el sistema solicita las coordenadas GPS del dispositivo y realiza una llamada asíncrona a la API de *OpenWeatherMap*.

La temperatura obtenida se normaliza y clasifica en tres categorías climáticas booleanas: Frío, Entretiempo o Calor. El segundo paso consiste en el filtrado del espacio de búsqueda, con el objetivo de garantizar que la recomendación sea útil y ejecutable, el algoritmo realiza una consulta a la base de datos para recuperar únicamente el subconjunto de prendas del usuario cuyo estado actual sea Limpio, En uso o A medias, según el usuario escoja.

La tercera fase del flujo consiste en la selección heurística combinatoria. Sobre el conjunto de prendas disponibles, el motor aplica diferentes reglas para formar una combinación coherente. El sistema selecciona pseudoaleatoriamente una prenda superior o un vestido, una prenda inferior y un calzado. Si la variable contextual indica Frío, el algoritmo añade dinámicamente una prenda de abrigo al conjunto.

El Algoritmo 5 recoge esta lógica. La pieza central es la función `ElegirCompatible`, que resuelve un hueco del conjunto en tres pasos: si el usuario ha fijado manualmente esa prenda, la respeta sin más; si no, recupera todas las candidatas de la categoría y se queda solo con las que combinan en color con las prendas ya elegidas. Cuando ningún candidato pasa el filtro cromático, el algoritmo recurre a una elección aleatoria sobre el conjunto completo, de modo que siempre devuelve un conjunto válido aunque el armario del usuario sea reducido. Sobre esa base, la primera fase decide entre vestido o combinación de superior e inferior, y la segunda añade la capa exterior en función del tipo de clima: una prenda de abrigo si hace frío, un jersey o una sudadera si es entretiempo, y ninguna si hace calor.

Por último, la persistencia y actualización de estado: una vez generado y validado el conjunto, el sistema ejecuta una transacción en la base de datos. Se crea un nuevo registro en la tabla *outfits*, vinculando las prendas en la tabla relacional *outfit_prendas* y, simultáneamente, se actualiza el estado de las prendas seleccionadas, pasando de Limpio a En uso para mantener la consistencia del inventario físico y digital.

Por último, queda la persistencia y la actualización de estado. Una vez generado y validado el conjunto, el sistema ejecuta una transacción en la base de datos, tal y como muestra el algoritmo 6. La operación consta de tres pasos que deben completarse en bloque. Primero se crea el registro del conjunto en la tabla *outfits*, con un nombre generado a partir del evento

Algorithm 5 Selección heurística combinatoria de prendas

Require: *pool* (Inventario disponible), *clima* (Contexto climático), *slotFijo* (Fijadas por el usuario)

Ensure: Combinación base generada (*superior*, *inferior*, *abrigo*)

```

1: function ELEGIRCOMPATIBLE(slot)
2:   if slotFijo[slot]  $\neq$  Nulo then
3:     return slotFijo[slot]
4:   end if
5:   candidatos  $\leftarrow$  OBTENERPRENDASPORCATEGORIA(slot)
       $\triangleright$  Filtramos asegurando que la prenda sea compatible con todas las ya elegidas
6:   compatibles  $\leftarrow$   $\{p \in \text{candidatos} \mid \forall e \in \text{yaElegidas}, \text{COLORCOMPATIBLE}(p, e)\}$ 
7:   if  $|\text{compatibles}| > 0$  then  $\triangleright$  Comprueba si la cardinalidad  $> 0$ 
8:     return ALEATORIO(compatibles)
9:   else
10:    return ALEATORIO(candidatos)  $\triangleright$  Fallback si no hay compatibilidad estricta
11:   end if
12: end function

```

$\triangleright$ Fase 1: Selección de prenda base (Superior o Vestido)

```

13: if puedeVestido and (not puedeConjunto or RANDOM  $> 0,5$ ) then
14:   superior  $\leftarrow$  ALEATORIO(vestidosPool)
15: else
16:   superior  $\leftarrow$  ALEATORIO(superioresPool)
17:   infCompat  $\leftarrow$   $\{p \in \text{inferioresPool} \mid \forall e \in \text{yaElegidas}, \text{COLORCOMPATIBLE}(p, e)\}$ 
18:   if  $|\text{infCompat}| > 0$  then
19:     inferior  $\leftarrow$  ALEATORIO(infCompat)
20:   else
21:     inferior  $\leftarrow$  ALEATORIO(inferioresPool)
22:   end if
23: end if

```

$\triangleright$ Fase 2: Selección de capa exterior basada en contexto térmico

```

24: if clima.tipo = 'Frío' then
25:   abrigo  $\leftarrow$  ELEGIRCOMPATIBLE('abrigo')
26: else if clima.tipo = 'Entretiempo' then
27:   capasLigeras  $\leftarrow$   $\{p \in \text{pool} \mid \text{'Jersey'} \in p.\text{categoria} \text{ or } \text{'Sudadera'} \in p.\text{categoria}\}$ 
28:   abrigo  $\leftarrow$  ALEATORIO(capasLigeras)
29: end if

```

y del clima. Después se vinculan las prendas mediante una inserción por lotes en la tabla *outfit_prendas*, que materializa la relación N:M entre conjuntos y prendas. Finalmente se actualiza el estado de las prendas seleccionadas, que pasan de Limpio a En uso. Este último paso es el que mantiene la consistencia entre el inventario físico y el digital: una prenda que forma parte de un conjunto activo deja de estar disponible para futuras recomendaciones hasta que el usuario la lave.

La figura 4.3 representa el flujo completo del motor de generación de conjuntos, organi-

Algorithm 6 Persistencia transaccional del conjunto generado

Require: *user.id* (ID de sesión activa), *evento* (Propósito del conjunto), *climaActual* (Contexto climático), *prendasElegidas* (Lista de identificadores)

Ensure: Consistencia del nuevo conjunto en la base de datos y actualización de inventario

```

1: function GUARDAROUTFITGENERADO(user.id, evento, climaActual, prendasElegidas)
    ▷ Paso 1: Crear el registro principal del conjunto

2:   nombreOutfit  $\leftarrow$  'Look' + evento + '-' + climaActual.tipo
3:   nuevoOutfit  $\leftarrow$  INSERTAR('outfits') con valores:
       id_usuario  $\leftarrow$  user.id
       nombre  $\leftarrow$  nombreOutfit
       clima_ideal  $\leftarrow$  climaActual.tipo
       evento_ideal  $\leftarrow$  evento
    ▷ Paso 2: Vincular las prendas en tabla de ruptura (Relación N:M)

4:   inserciones  $\leftarrow$   $\emptyset$ 
5:   for all id_prenda  $\in$  prendasElegidas do
6:     inserciones  $\leftarrow$  inserciones  $\cup$  {(nuevoOutfit.id, id_prenda)}
7:   end for
8:   INSERTARLOTE('outfit_prendas', inserciones)
    ▷ Paso 3: Actualizar el estado físico de la ropa en el inventario

9:   ACTUALIZAR('prendas') con valores:
       estado  $\leftarrow$  'En uso'
    donde id  $\in$  prendasElegidas
10: end function

```

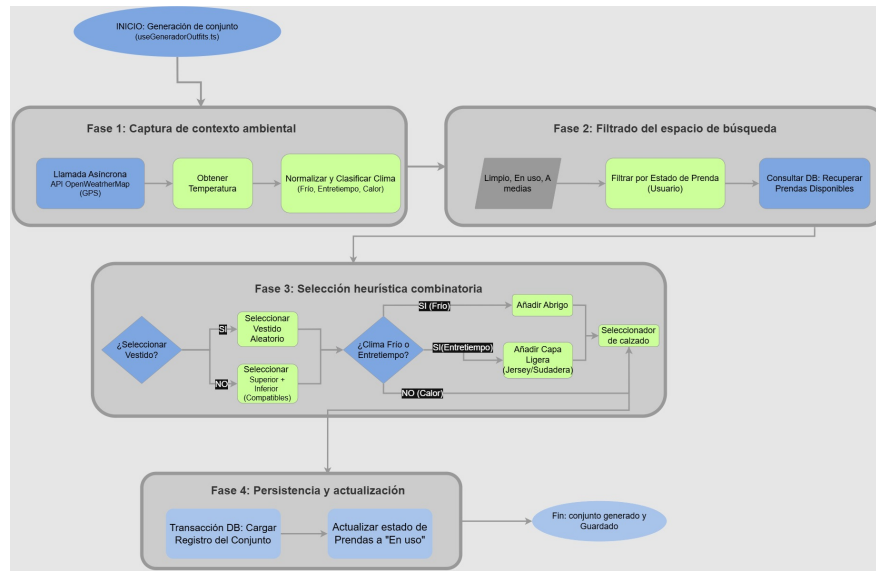


Figura 4.3: Flujo del sistema de recomendación sensible al contexto

zado en cuatro fases: la captura del contexto ambiental, el filtrado de prendas disponibles según su estado, la selección heurística combinatoria basada en clima y compatibilidad, y la persistencia final del conjunto en la base de datos con la actualización del estado de las prendas.

4.3.3 Algoritmo de organización y reglas de almacenamiento

El tercer flujo crítico aborda la digitalización del espacio físico del armario. Para lograr la máxima la durabilidad de la ropa y optimizar el espacio disponible configurado por el usuario en la tabla `armario_config`, el sistema implementa un motor de reglas de almacenamiento (`reglasAlmacenamiento.ts`). Este módulo actúa como un sistema experto de clasificación espacial que evalúa cada prenda ingresada basándose en dos dimensiones: su categoría, es decir, el tipo de prenda que es, si es un vestido, una camisa o un pantalón; y su composición textil extraída previamente por el microservicio OCR. El sistema experto devuelve una estructura de datos normalizada, detallada en el algoritmo 7, que encapsula el método de almacenamiento recomendado junto con consejos y alertas para el usuario.

Algorithm 7 Interfaz de salida del sistema experto de almacenamiento

Estructura *Sugerencia*

- 1: *metodo* $\in \{ \text{'colgar'}, \text{'doblar'}, \text{'enrollar'}, \text{'zapatero'} \}$
- 2: *icono* $\leftarrow$ Cadena de texto
- 3: *tipoPercha* $\in \{ \text{'estándar'}, \text{'ancha'}, \text{'con pinzas'}, \text{'acolchada'} \}$ ▷ Atributo opcional
- 4: *zona* $\leftarrow$ Cadena de texto (Ubicación física)
- 5: *consejo* $\leftarrow$ Cadena de texto (Recomendación razonada)
- 6: *alerta* $\leftarrow$ Cadena de texto (Aviso de capacidad) ▷ Atributo opcional

Fin Estructura

El flujo de procesamiento para este módulo tiene varias partes. Primero se evalúan las reglas de negocio: el sistema aplica un árbol de decisión, formalizado en el algoritmo 8, para determinar el método de almacenamiento óptimo de cada prenda. Por ejemplo, los algoritmos dictaminan que la categoría *Abrigos* debe asignarse al estado *Colgar* y sugieren una percha ancha, mientras que los *Jerseys* o prendas con un alto porcentaje de lana se fuerzan al estado *Doblar* con el objetivo de evitar la deformación por gravedad del tejido. Las prendas más informales o de menor volumen como pueden ser los *Tops*, se derivan al estado *Enrollar* para optimizar el espacio.

El árbol de decisión anterior se aplica de forma iterativa sobre la totalidad del inventario, tal como recoge el Algoritmo 9, agrupando cada prenda en su contenedor lógico correspondiente. Una vez agrupado el inventario en los cuatro contenedores lógicos (Colgar, Doblar, Enrollar, Zapatero), el sistema cruza estos volúmenes con las métricas físicas definidas por el usuario: número de barras, baldas, cajones y zapatero. Si el algoritmo detecta que el número de prendas excede la capacidad geométrica teórica del armario, emite una alerta visual preventiva en la interfaz, promoviendo la optimización del espacio.

4.3.4 Planificador inteligente de coladas (optimización del lavado)

El último flujo principal del sistema resuelve el problema logístico del mantenimiento de la ropa, minimizando el riesgo de estropear tejidos por mezclas incorrectas de colores o temperaturas. Cuando el usuario decide planificar una colada, el sistema ejecuta un algoritmo

Algorithm 8 Árbol de decisión del motor de reglas de almacenamiento**Require:** *cat* (Categoría), *tela* (Composición), *formal* (Booleano)**Ensure:** Objeto *Sugerencia* con método y zona óptima

```

1: function DETERMINARALMACENAMIENTO(cat, tela, formal)
    ▷ Caso 1: Prendas de abrigo con estructura
2:   if 'Abrigo' ∈ cat or 'Chaqueta' ∈ cat then
3:     res ← Crear Sugerencia (metodo: 'colgar', tipoPercha: 'ancha')
4:     res.zona ← 'Barra de perchas'
5:     res.consejo ← 'Preservar la forma del hombro'
6:     if tela = 'Cuero' then
7:       res.alerta ← 'Evitar fundas plásticas; requiere transpiración'
8:     end if
9:     return res
10:  end if
    ▷ Caso 2: Punto y tejidos elásticos
11:  if 'Jersey' ∈ cat or 'Sudadera' ∈ cat then
12:    res ← Crear Sugerencia (metodo: 'doblar', zona: 'Balda o Cajón')
13:    res.consejo ← 'Doblar en 3 partes; apilado máximo de 4'
14:    if tela = 'Lana' then
15:      res.alerta ← 'NUNCA colgar: el peso de la gravedad deforma el tejido'
16:    end if
17:    return res
18:  end if
    ▷ Caso 3: Prendas informales de bajo volumen
19:  if 'Top' ∈ cat or 'Camiseta' ∈ cat then
20:    if not ESTELAFINA(tela) and not formal then
21:      res ← Crear Sugerencia (metodo: 'enrollar', zona: 'Cajón')
22:      res.consejo ← 'Método KonMari: ahorro de espacio y visibilidad vertical'
23:      return res
24:    end if
25:  end if
26: end function

```

Algorithm 9 Aplicación del motor de reglas sobre el inventario completo**Require:** *inventario* (Lista completa de prendas obtenida de la base de datos)**Ensure:** *contenedores* (Estructura agrupada por métodos de almacenamiento)

```

1: contenedores ← Iniciar diccionario vacío con listas para cada método (colgar, doblar, enrollar, zapatero)
2: for all prenda ∈ inventario do
3:   sugerencia ← DETERMINARALMACENAMIENTO(prenda)
4:   contenedores[sugerencia.metodo].AÑADIR(prenda, sugerencia)
5: end for
6: return contenedores

```

de, Clustering basado en reglas:

1. Identificación de la carga de trabajo: el sistema filtra el inventario para aislar exclusivamente las prendas cuyo estado actual sea *Sucio* o *A medias*. Este primer filtrado, que

4. SOLUCIÓN PROPUESTA Y HERRAMIENTAS

separa además las prendas de lavado a mano de las aptas para lavadora, se detalla en el Algoritmo 10.

Algorithm 10 Separación inicial por método de lavado permitido

Require: *prendas, simbolosPorPrenda*

Ensure: Listas filtradas y clasificadas (*soloMano, aMaquina*)

```
1: for all prenda  $\in$  prendas do
2:   simbolos  $\leftarrow$  simbolosPorPrenda[prenda.id]
3:   if simbolos = Nulo then
4:     simbolos  $\leftarrow$   $\emptyset$ 
5:   end if
6:   // Restricciones críticas de lavado
7:   noAgua  $\leftarrow \exists s \in \text{simbolos} : s.\text{codigo} \in \{\text{NO\_LAVAR, LIMPIEZA\_SECO}\}$ 
8:   soloManoSim  $\leftarrow \exists s \in \text{simbolos} : s.\text{codigo} = \text{LAVADO\_MANO}$ 
9:   telaManual  $\leftarrow$  (prenda.tipo_tela = Cuero)
10:  // Direccionamiento de la prenda
11:  if noAgua then
12:    continue
13:  else if soloManoSim  $\vee$  telaManual then
14:    soloMano.AÑADIR(prenda)
15:  else
16:    aMaquina.AÑADIR(prenda)
17:  end if
18: end for
```

2. Segmentación bidimensional: El conjunto de ropa sucia se divide de forma iterativa en subgrupos utilizando dos variables críticas. En la primera pasada, se separa por colorimetría: blancos, oscuros y color. En la segunda pasada, cada subgrupo de color se vuelve a dividir según la fragilidad de la composición textil Delicados, Sintéticos y Algodón/Normal, basándose en las propiedades que el microservicio OCR detectó al leer la etiqueta. El Algoritmo 11 formaliza esta segmentación bidimensional.
3. Cálculo de restricciones operativas: Para cada colada generada el algoritmo escanea la temperatura máxima permitida de cada prenda individual que la compone. El sistema calcula la temperatura mínima de seguridad del conjunto, por ejemplo, si hay tres prendas de cuarenta grados centígrados y una de treinta grados centígrados, el sistema fuerza la colada entera a treinta grados centígrados para garantizar la integridad de todos los tejidos. El Algoritmo 12 describe este cálculo de la temperatura mínima de seguridad
4. Validación de hardware: Finalmente, el peso estimado de las coladas sugeridas se contrasta con la capacidad volumétrica (*carga_maxima_kg*) del electrodoméstico registrado por el usuario en la tabla *catalogo_lavadoras*, sugiriendo divisiones si se excede la carga máxima soportada.

Algorithm 11 Agrupamiento bidimensional por color y fragilidad textil

Require: *prenda* (Objeto a clasificar), *simbolos* (Lista de símbolos OCR), *grupos* (Diccionario de agrupamiento global)**Ensure:** Prenda asignada al contenedor correspondiente dentro de *grupos*1: *colorBucket* $\leftarrow$ OBTENERCATEGORIACOLOR(*prenda*) $\triangleright$ Retorna ‘blancos’, ‘oscuros’ o ‘color’ $\triangleright$ Paso 1: Evaluación de fragilidad y composición textil2: *simboloDelicado* $\leftarrow \exists s \in \text{simbolos tal que } s.es_delicado = \text{true}$ 3: *esDelicado* $\leftarrow$ *prenda.es_delicado* **or** *simboloDelicado* **or** (*prenda.tipo_tela* $\in$ TELAS_DELICADAS)4: *esSintetico* $\leftarrow$ (**not** *esDelicado*) **and** (*prenda.tipo_tela* $\in$ TELAS_SINTETICAS) $\triangleright$ Paso 2: Clasificación del subgrupo térmico/mecánico5: **if** *esDelicado* **then**6: *subgrupo* $\leftarrow$ ‘delicado’7: **else if** *esSintetico* **then**8: *subgrupo* $\leftarrow$ ‘sintetico’9: **else**10: *subgrupo* $\leftarrow$ ‘normal’11: **end if** $\triangleright$ Paso 3: Generación de la clave compuesta y almacenamiento12: *clave* $\leftarrow$ *colorBucket* + ‘_’ + *subgrupo* $\triangleright$ Ejemplo: ‘oscuros_delicado’13: **if** *grupos*[*clave*] = Nulo **then**14: *grupos*[*clave*] $\leftarrow \emptyset$ $\triangleright$ Inicializar la lista si la clave no existe15: **end if**16: *grupos*[*clave*].AÑADIR(*prenda*)

Algorithm 12 Cálculo de la temperatura mínima de seguridad por colada

Require: *subgrupo* (Clasificación térmica), *prendasGrupo* (Conjunto de prendas en la colada), *simbolosPorPrenda* (Diccionario de símbolos de cuidado seleccionados por el usuario)

Ensure: *tempMax* (Temperatura máxima de seguridad en grados Celsius)

▷ Paso 1: Asignación de la temperatura base según el tipo de tejido

```

1: if subgrupo = 'delicado' then
2:   tempMax  $\leftarrow$  30
3: else if subgrupo = 'sintetico' then
4:   tempMax  $\leftarrow$  40
5: else
6:   tempMax  $\leftarrow$  60
7: end if

```

▷ Paso 2: Reducción heurística al mínimo tolerable por el conjunto

```

8: for all p  $\in$  prendasGrupo do
    ▷ Evaluación del límite térmico intrínseco de la prenda
9:   if p.temp_lavado > 0 and p.temp_lavado < tempMax then
10:    tempMax  $\leftarrow$  p.temp_lavado
11:   end if

```

▷ Evaluación de los límites dictados por las etiquetas (OCR)

```

12:  simbolos  $\leftarrow$  simbolosPorPrenda[p.id]
13:  if simbolos  $\neq$  Nulo then
14:    for all s  $\in$  simbolos do
15:      if s.temp_maxima  $\neq$  Nulo and s.temp_maxima < tempMax then
16:        tempMax  $\leftarrow$  s.temp_maxima
17:      end if
18:    end for
19:  end if
20: end for
21: return tempMax

```

Capítulo 5

Resultados

ESTE capítulo presenta los resultados obtenidos tras la implementación de *Armario-Virtual*, mostrando la aplicación en funcionamiento sobre datos reales. Este capítulo presenta dos objetivos, por un lado, ilustrar el aspecto final de cada pantalla y el flujo de interacción que sigue el usuario. Por otro lado, validar que las funcionalidades escritas en los capítulos anteriores se han materializado y cumplen los objetivos planteados en un principio. Para ello se recorre la aplicación siguiendo el orden natural de uso, desde el acceso inicial hasta las funciones de organización más avanzadas, conectando cada resultado visual con la lógica y los algoritmos detallados. Todas las capturas se han tomado sobre la aplicación real ejecutándose en un dispositivo físico, con prendas y datos introducidos por un usuario, sin recurrir a datos simulados.

5.1 Acceso y registro de usuarios

El primer contacto del usuario con la aplicación se produce a través de la pantalla de inicio de sesión, que junto al formulario de registro, se muestra en la figura 5.1. La aplicación se apoya en el sistema de autenticación de Supabase, de modo que las credenciales se validan de forma segura contra el backend y se establece una sesión persistente. Un usuario que ya dispone de cuenta introduce su correo electrónico y contraseña para acceder directamente a la pantalla principal.

En caso de tratarse de un usuario nuevo, la opción de crear una cuenta conduce al formulario de registro, que recoge tanto los datos de acceso, es decir, el correo y la contraseña, como la información básica de perfil: nombre de usuario, nombre, apellidos y teléfono. Esta separación entre datos de acceso y datos de perfil refleja la estructura de la tabla de usuarios descrita en capítulos anteriores, donde la identidad de autenticación se vincula con la información personal asociada. Una vez completado el registro, el usuario queda automáticamente autenticado y es dirigido al interior de la aplicación, evitando así un segundo paso de inicio de sesión.

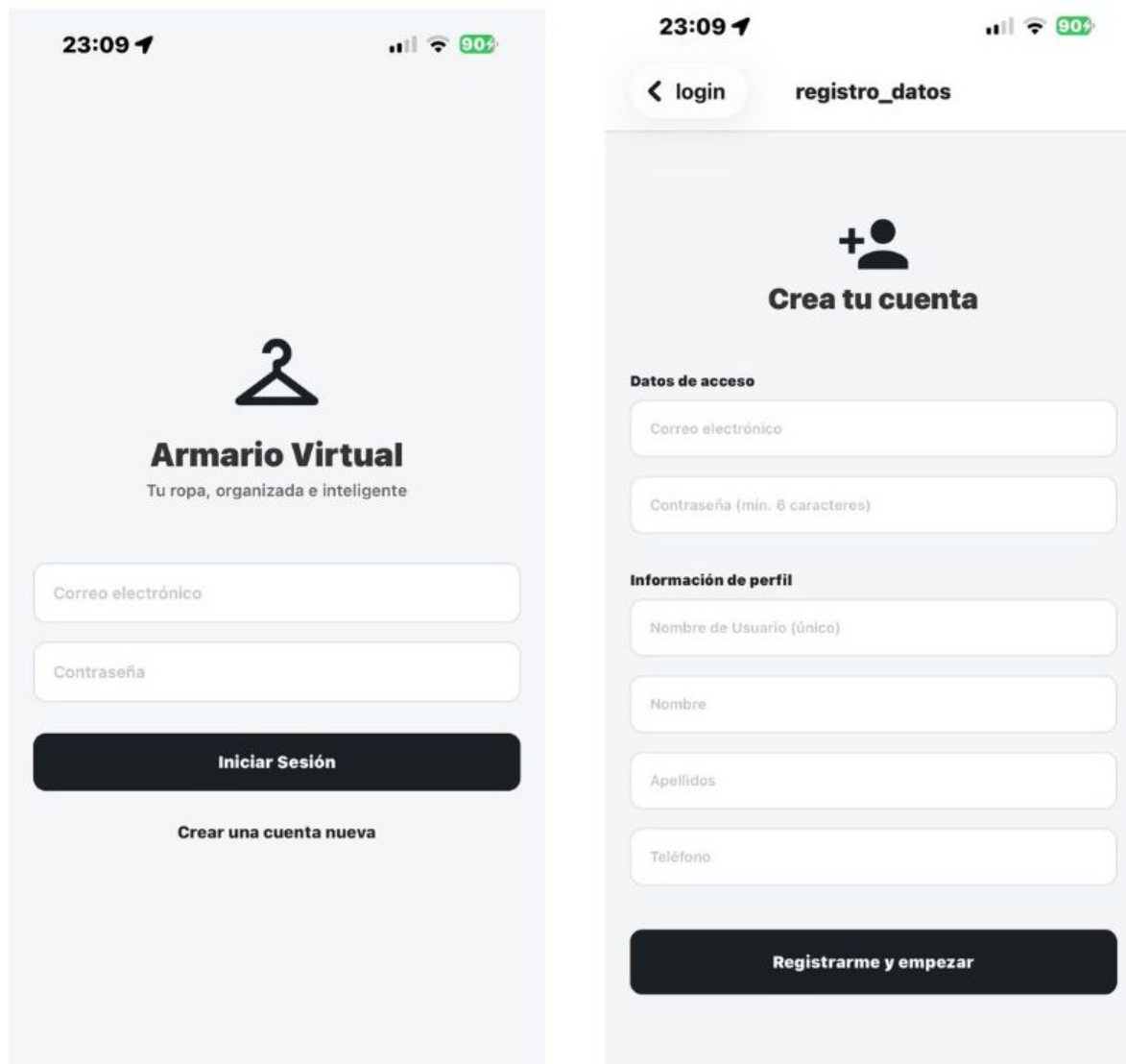


Figura 5.1: Inicio de Sesión y Creación de Usuarios

5.2 Digitalización de prendas

La incorporación de prendas al armario constituye la funcionalidad central de la aplicación y el punto de partida sobre el que operarán los diferentes módulos del sistema. Sabiendo que esta acción es la más frecuente durante las primeras fases de uso, se ha diseñado un doble punto de acceso para facilitar al usuario la adición de prendas, pudiendo hacerlo desde la pantalla de inicio y desde la pantalla de las propias prendas, tal como se muestra en la figura 5.2.

Esta pantalla Prendas, actúa además como catálogo visual del armario completo. Las prendas se muestran en una cuadrícula de dos columnas en la que cada imagen aparece recortada sobre un fondo transparente, resultado del procesamiento de eliminación de fondo. Sobre este catálogo se ofrece un sistema de filtrado por categorías: favoritos, tops, pantalones, etc.

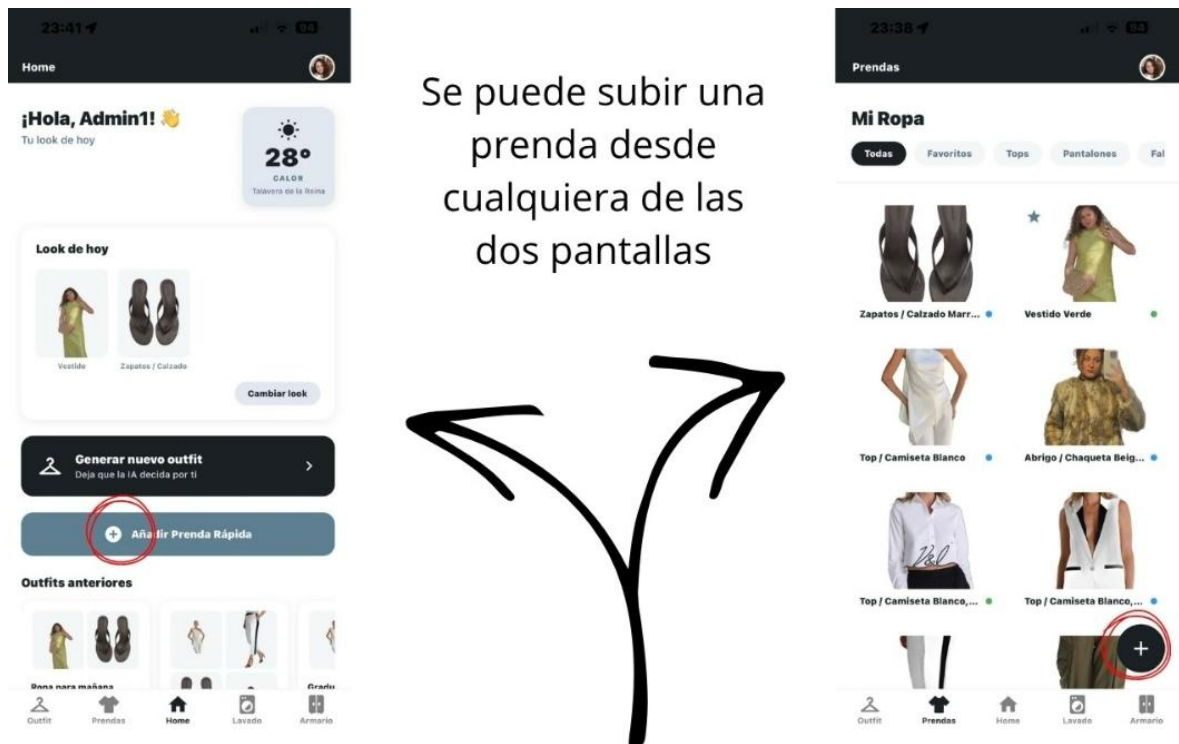


Figura 5.2: Pantalla de registro de prenda

todo esto, permite localizar de forma rápida cualquier prenda a medida que la colección crece. Cada elemento incluye un indicador de color y su descripción, generada según la información introducida al introducir la prenda, que facilita el reconocimiento de la prenda sin necesidad de abrir la ficha completa.

5.2.1 Proceso de digitalización

Una vez iniciado el proceso, la aplicación guía al usuario a través de la secuencia de pasos que debe seguir para subir la prenda. En primer lugar, se ofrece la elección del origen de la imagen, permitiendo capturar una fotografía en el momento con la cámara o seleccionar una imagen existente de la galería. A continuación, el usuario asigna la categoría de la prenda, top, camisa, pantalón, falda, etc. El siguiente paso permite indicar los colores que tiene la prenda mediante una lista desplazable.

El elemento más destacado de este proceso es el tratamiento automático de la imagen. Tras seleccionar la fotografía de la prenda, esta se envía al microservicio de procesamiento descrito anteriormente, donde se elimina el fondo de forma automática y se devuelve una imagen recortada con transparencia. Paralelamente, el usuario puede fotografiar la etiqueta de la prenda para su posterior análisis. El sistema de reconocimiento óptico de caracteres (OCR) procesa dicha imagen y extrae la información de composición del tejido a partir del texto impreso en la etiqueta.

5. RESULTADOS

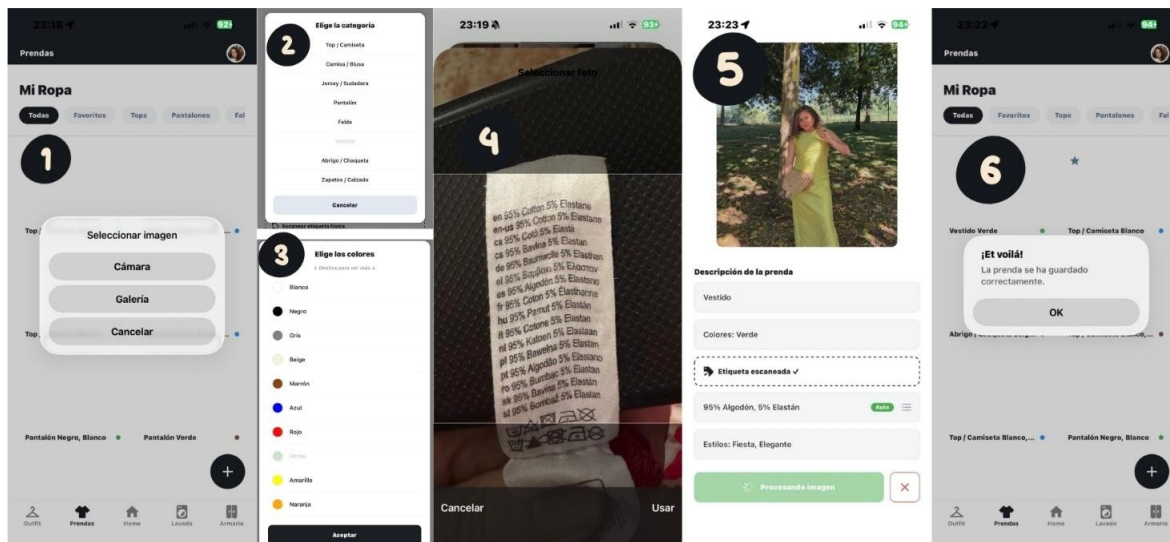


Figura 5.3: Proceso de subir una prenda

Como se aprecia en la captura 5.3, el sistema identifica correctamente la composición «95 % Algodón, 5 % Elastán» a partir de una etiqueta que repite dicha información en múltiples idiomas, y la marca como detectada de forma automática. Finalmente, el usuario completa la información asignando los estilos que mejor describen la prenda: diario, casual, etc. y confirma el guardado, tras lo cual la prenda pasa a formar parte del catálogo y queda inmediatamente disponible en la pantalla *Prendas*.

Conviene apuntar que el reconocimiento óptico opera exclusivamente sobre el texto de la etiqueta. La lectura automática de los símbolos normalizados de cuidado, los pictogramas de lavado, secado y planchado, queda fuera del alcance de esta técnica, y se resuelve mediante un mecanismo de selección asistida que se describe más adelante.

5.3 Ficha de prendas y gestión de estados

Cada prenda del catálogo dispone de una ficha detallada que muestra toda su información y permite gestionar su ciclo de vida dentro del armario, tal como se muestra en la figura 5.4. En la parte superior se muestra la imagen recortada de la prenda junto con su denominación, y se ofrece la posibilidad de marcarla como favorita. El apartado de estado permite reflejar la situación real de la prenda mediante cuatro valores: limpio, a medias, en uso y sucio. Toda esta información después se utilizará en el módulo de lavandería y para el generador de conjuntos, que puede así excluir prendas que no estén disponibles. Bajo el estado se muestran los atributos descriptivos de la prenda: categoría, color, composición del tejido y estilos asociados.

En la parte inferior de la ficha se encuentra la gestión de los símbolos de cuidado. Dado que estos pictogramas no pueden extraerse automáticamente de la etiqueta, la aplicación ofrece

un catálogo visual de símbolos con ciclos de temperatura, instrucciones de secado, limpieza en seco, entre otros. Por lo que, el usuario selecciona aquellos que aparecen en la etiqueta de su prenda. Cada símbolo se acompaña de una descripción textual y de su categoría, lo que facilita su identificación incluso a usuarios no familiarizados con la simbología textil. Una vez seleccionados, los símbolos quedan asociados a la prenda y pueden editarse en cualquier momento. Este enfoque combina el reconocimiento automático del texto con la aportación manual del usuario para los pictogramas, garantizando que la información de cuidado sea completa y fiable sin depender de una tecnología de visión que excedería el alcance del proyecto.

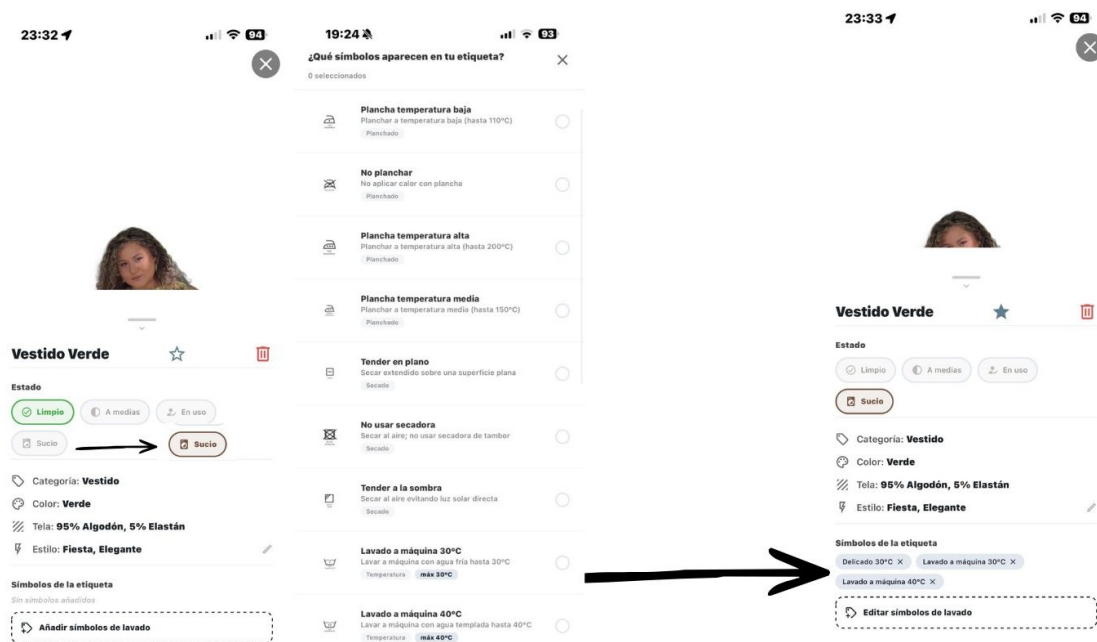


Figura 5.4: Gestión de las características de la prenda

5.4 Edición y eliminación de prendas

La gestión del catálogo se completa con la posibilidad de eliminar prendas que ya no forman parte del armario del usuario. La acción de borrado solicita una confirmación explícita, advirtiendo de que se eliminarán tanto los datos de la prenda como su imagen asociada de forma definitiva, lo que previene borrados accidentales como se aprecia en la figura 5.5. Tras confirmar, la prenda desaparece del catálogo y se notifica al usuario que la operación se ha completado correctamente. El catálogo, refleja en todo momento el estado actual del armario, actualizándose de forma inmediata ante cualquier alta o baja de prendas.

5. RESULTADOS

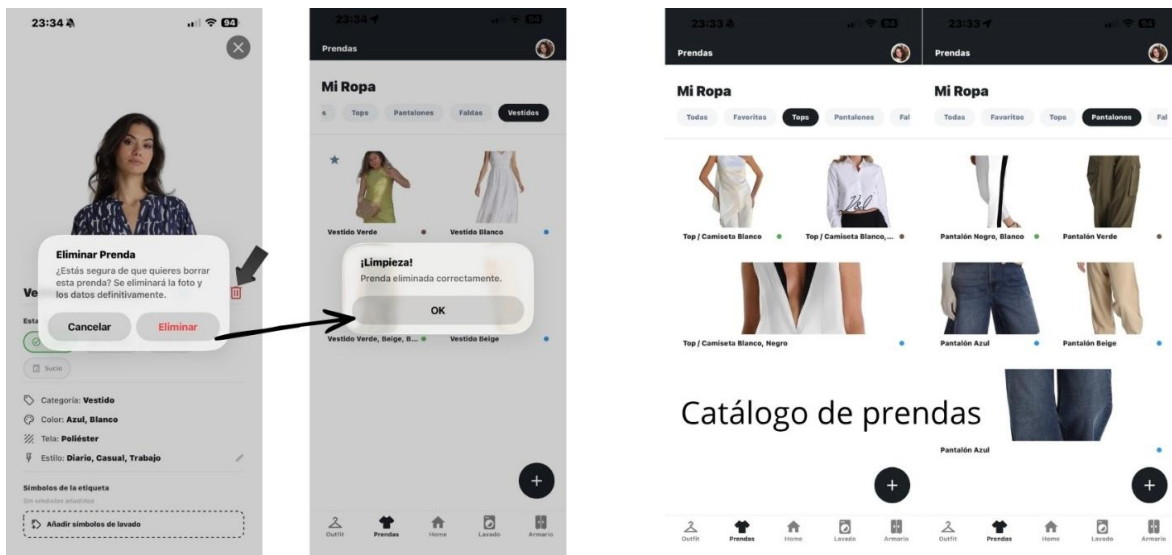


Figura 5.5: Visualización pantalla prendas y eliminación de prendas

5.5 Generación de conjuntos por contexto

El generador de conjuntos representa la funcionalidad de mayor valor añadido de la aplicación, al automatizar la decisión sobre cómo vestir un día u ocasión a partir del contexto del usuario. El proceso comienza con la determinación de la temperatura actual de la ubicación del usuario, que puede obtenerse automáticamente mediante geolocalización o introducirse manualmente cuando el usuario desea preparar un conjunto para una ubicación o momento distintos del actual. A continuación, el usuario indica la ocasión para la que necesita el conjunto mediante las categorías de estilo disponibles.

Antes de generar el conjunto, la aplicación plantea una consulta que enriquece la experiencia: ofrece la posibilidad de completar el conjunto con prendas que el usuario ya lleve puestas, de generarlo sin restricciones adicionales, o de limitar la selección exclusivamente a prendas limpias. Esta última opción conecta directamente con el sistema de estados descrito anteriormente. Una vez procesada la solicitud, el motor de reglas compone un conjunto coherente, presentando las prendas seleccionadas organizadas según sean parte superior, parte inferior, vestido, abrigo o capa, o calzado en una visualización limpia en la que cada prenda se muestra sobre fondo transparente sin superponerse a las demás como se observa en la figura 5.6. La inclusión de una prenda de abrigo responde a la lógica de la temperatura detectada, de modo que un clima frío activa la incorporación de una capa térmica al conjunto.

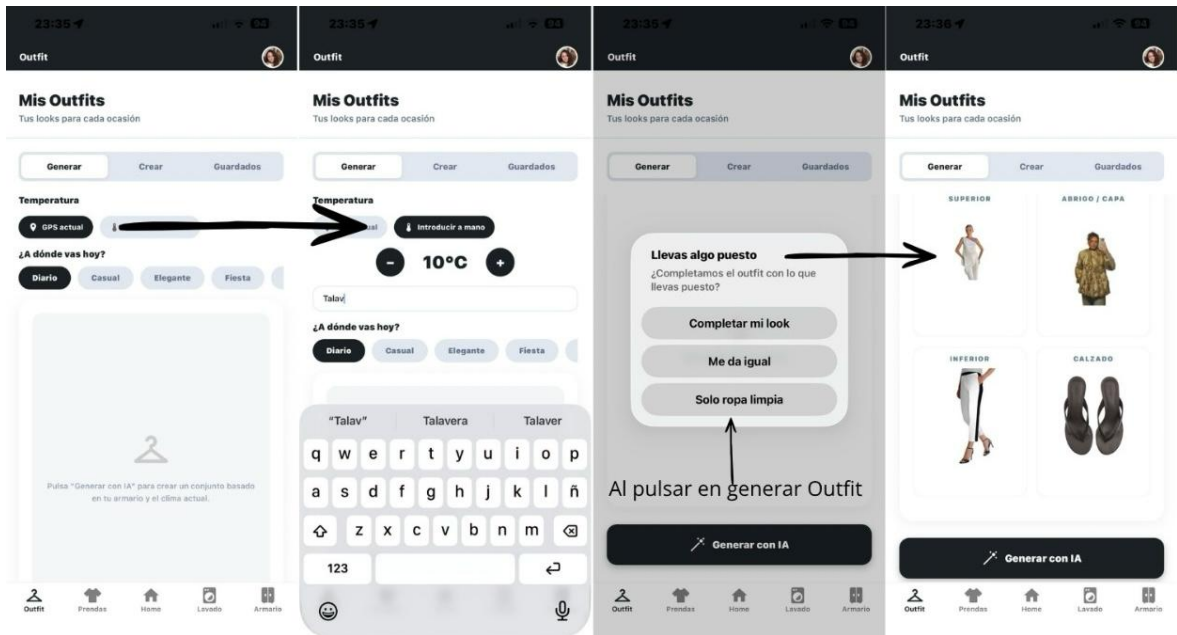


Figura 5.6: Generar un conjunto pseudoaleatoriamente

5.6 Guardado y creación manual de conjuntos

Los conjuntos generados pueden guardarse en la colección personal del usuario para su consulta posterior. Junto a la generación automática, la aplicación contempla igualmente la creación manual de conjuntos, en la que el usuario selecciona directamente las prendas que desea combinar y les asigna una ocasión y un nombre descriptivo. La figura 5.7 muestra este proceso de creación manual. Los conjuntos guardados se organizan en una galería propia, etiquetados por ocasión y clima, desde la que pueden consultarse o eliminarse.

Adicionalmente, cada conjunto puede exportarse como una imagen compuesta que reúne todas sus prendas en una única lámina identificada con el nombre del look, lo que permite al usuario conservarla en la galería de su dispositivo o compartirla con facilidad.

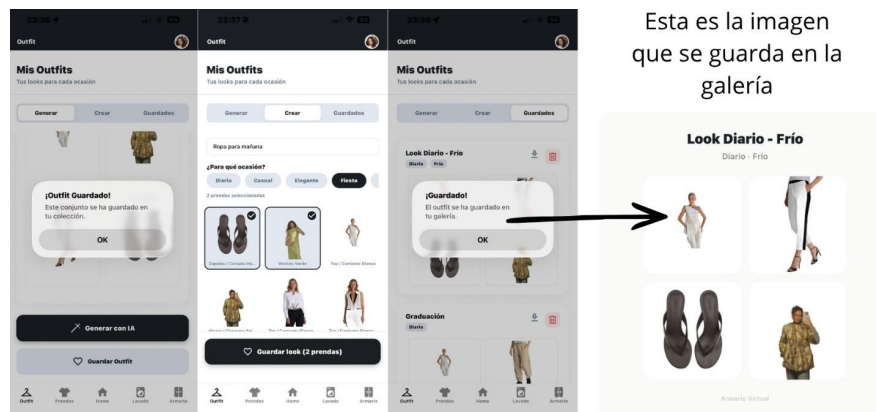


Figura 5.7: Creación manual de un conjunto

5. RESULTADOS

5.7 Lavandería inteligente

El módulo de lavandería materializa una de las aportaciones más originales de la aplicación, al transformar los datos de cuidado de las prendas en recomendaciones sobre cómo hacer la colada. Partiendo de la configuración de la lavadora del usuario, el sistema agrupa automáticamente las prendas que se encuentran pendientes de lavado en coladas compatibles, atendiendo a criterios de color y tipo de tejido.

Cada colada recomendada se presenta, como puede verse en la figura 5.8, con el programa de lavado sugerido: delicado, algodón o sintéticos; y la temperatura correspondiente, que es calculada como la temperatura mínima de seguridad del conjunto de prendas que la componen. De este modo se garantiza que ninguna prenda de la colada se lave a una temperatura superior a la que su etiqueta permite.

El usuario puede confirmar la realización de una colada mediante una acción explícita. Al confirmar el lavado, el estado de las prendas implicadas se actualiza automáticamente a «limpio» en el catálogo, como refleja el cambio del indicador correspondiente en la pantalla Prendas. Esta actualización cierra el ciclo entre el módulo de lavandería y la gestión de estados, de manera que la información del armario permanece siempre coherente con la situación real de las prendas y el generador de conjuntos dispone de datos actualizados sobre su disponibilidad.

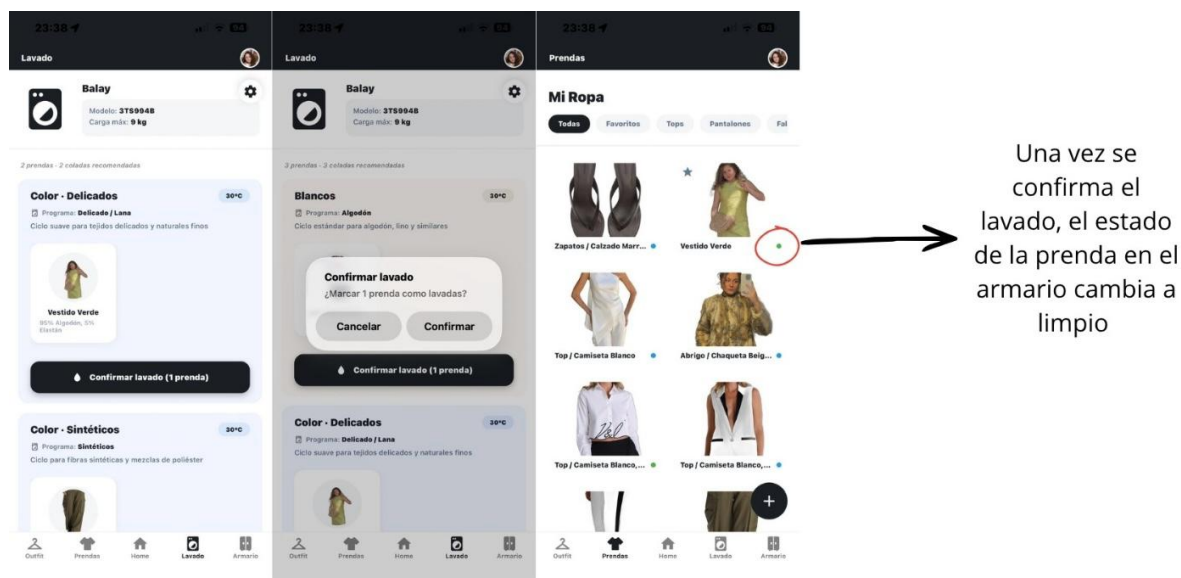


Figura 5.8: Funcionalidad del lavado de prendas

5.8 Configuración de la lavadora y perfil de usuario

Para que las recomendaciones de lavado se ajusten al electrodoméstico concreto del usuario, la aplicación permite configurar los datos de la lavadora, como se aprecia en la figura 5.9. El

usuario puede seleccionar su modelo a partir de un catálogo de marcas y modelos comerciales que ya tienen la información predeterminada de los parámetros de la lavadora como la carga máxima, la temperatura máxima, la velocidad de centrifugado y la clase energética; o bien introducir estos datos manualmente si su modelo no figura en el catálogo. Estos parámetros condicionan las recomendaciones del módulo de lavandería descrito anteriormente.

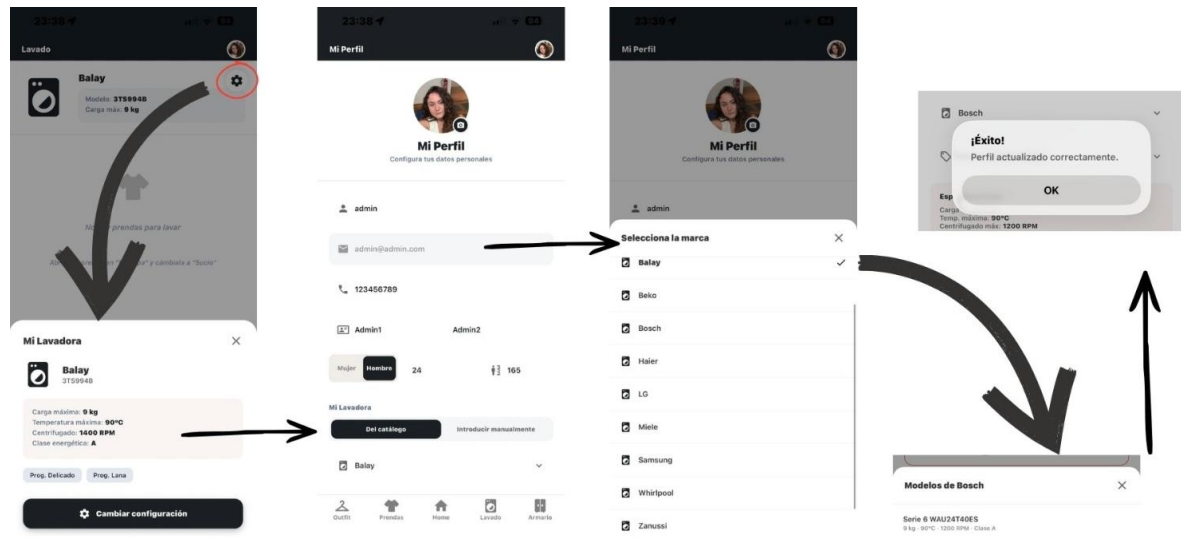


Figura 5.9: Configuración de la lavadora y perfil de usuario

5.9 Organización del armario físico

La última funcionalidad de la aplicación aborda la organización espacial del armario físico del usuario. A partir de una configuración del espacio disponible rellenando el número de barras que tiene el armario, el de perchas, la longitud de cada barra, el número de baldas y cajones, y la disponibilidad de zapatero.

El sistema clasifica automáticamente cada prenda del armario y recomienda su ubicación óptima. Esta clasificación atiende al tipo de tejido y a la categoría de la prenda para decidir entre colgar en percha, doblar en cajón o balda, o almacenar en el zapatero, con el objetivo de preservar la forma de cada prenda y evitar tanto arrugas como deformaciones.

La interfaz presenta las prendas agrupadas por tipo de almacenamiento, indicando para cada una la ubicación concreta recomendada y una breve justificación de la recomendación, como puede ser preservar la forma del hombro, en el caso de las prendas colgadas o evitar deformaciones, en los tejidos de punto. Un panel resumen informa además del grado de ocupación de cada zona de almacenamiento, ayudando al usuario a dimensionar su espacio. La figura 5.10 recoge esta pantalla de organización junto con la opción de cierre de sesión.

Desde la sección de perfil, el usuario puede gestionar sus datos personales y el cierre de sesión, que solicita confirmación antes de abandonar la aplicación.

5. RESULTADOS

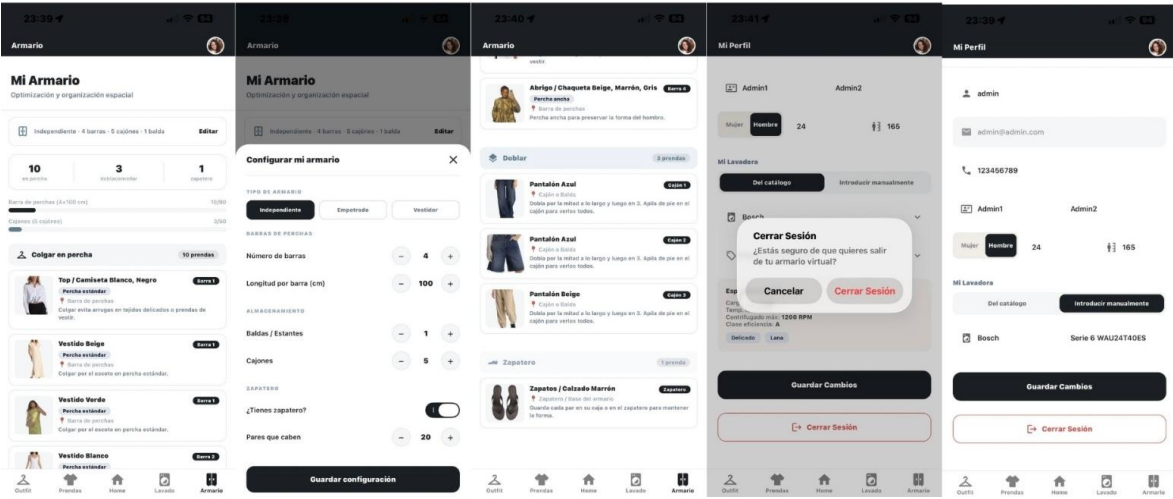


Figura 5.10: Organización del armario y Cierre de sesión

Capítulo 6

Conclusiones

EL presente Trabajo de Fin de Grado ha abordado el diseño, desarrollo e implementación de *ArmarioVirtual*, una aplicación móvil concebida como un asistente personal de vestuario que trasciende el concepto de inventario digital estático.

A lo largo de esta memoria se ha documentado el ciclo de vida completo del sistema desde la identificación de una problemática cotidiana, la carga cognitiva y logística asociada a la gestión del vestuario, hasta la validación funcional de una solución.

Este capítulo final recapitula el grado de consecución de los objetivos planteados, reflexiona sobre las competencias movilizadas y las limitaciones encontradas, y traza las líneas de trabajo futuro que el proyecto deja abiertas.

6.1 Grado de consecución de los objetivos

El objetivo general del proyecto, el desarrollo de una aplicación móvil para gestionar de forma virtual el armario de un usuario que se encargue de automatizar la digitalización del inventario y asistir al usuario en la selección de conjuntos y en el mantenimiento de sus prendas, es considerado alcanzado de forma satisfactoria. La aplicación resultante es un sistema funcional y completo que opera sobre datos reales, tal como se ha evidenciado en el capítulo de resultados.

En cuanto al sistema de digitalización automatizada, se ha implementado con éxito un módulo que incorpora prendas al inventario mediante fotografía y elimina automáticamente el fondo de la imagen a través de un microservicio de procesamiento, devolviendo una representación visual limpia y recortada de cada artículo. Este objetivo no solo se ha cumplido, sino que se ha complementado con un sistema de reconocimiento óptico de caracteres capaz de extraer la composición textil de las etiquetas de cuidado, dotando al sistema de una capacidad de captura de datos superior a la inicialmente prevista.

El motor de recomendación de conjuntos se ha traducido en un generador que combina el clima actual del usuario, obtenido por geolocalización, con las prendas que este tiene disponibles en el armario, aplicando reglas de compatibilidad y de temperatura. El sistema crea conjuntos coherentes y, cuando el armario no permite una combinación perfecta, es

6. CONCLUSIONES

capaz de adaptarse para ofrecer igualmente una propuesta útil, cumpliendo así el objetivo planteado.

El módulo de lavandería inteligente agrupa automáticamente las prendas pendientes de lavado en coladas compatibles, atendiendo a criterios de color y composición textil, y calcula para cada colada la temperatura mínima de seguridad que preserva la integridad de todas las prendas que la componen. La integración de este módulo con la configuración del electrodoméstico del usuario y con el sistema de estados de las prendas cierra un ciclo funcional completo, cumpliendo el objetivo definido.

El motor de optimización espacial genera recomendaciones personalizadas de almacenamiento para cada prenda en función de su categoría y composición textil, distinguiendo entre las prendas que deben colgarse, doblarse o guardarse para preservar su forma y evitar su deterioro.

Finalmente, el objetivo relativo a la interfaz de usuario intuitiva y multiplataforma se ha satisfecho mediante el desarrollo de una experiencia coherente y accesible que integra todas las funcionalidades anteriores sobre una base tecnológica única para las plataformas móviles principales.

6.2 Aportaciones y competencias adquiridas

Más allá del cumplimiento de los objetivos funcionales, la principal aportación de este trabajo reside en la integración coherente, dentro de una única plataforma, de funcionalidades que el análisis del estado del arte reveló como habitualmente ausentes o fragmentadas en las soluciones existentes.

Mientras que la mayoría de las aplicaciones de gestión de vestuario se limitan a ofrecer un inventario digital, *ArmarioVirtual* aporta inteligencia aplicada sobre ese catálogo a través de la recomendación sensible al contexto, la planificación de coladas y la optimización del espacio físico.

El desarrollo del proyecto ha exigido la adoptar e integrar varias de las competencias asociadas a la intensificación en Sistemas de Información del grado de Ingeniería Informática. A continuación, se detalla el modo en que el trabajo ha contribuido a reforzar cada una de ellas:

- **[CM5]** Los tres motores de inteligencia del sistema han supuesto trasladar a código el conocimiento experto sobre el cuidado textil recogido en las normativas de etiquetado. Las reglas de compatibilidad de colores, la agrupación de coladas, el cálculo de la temperatura de lavado segura y las reglas de almacenamiento son la traducción de ese conocimiento humano a algoritmos. Además, el sistema percibe su entorno y actúa en consecuencia, captando el clima por geolocalización y leyendo las etiquetas mediante

visión por computador para adaptar sus resultados a la situación real del usuario.

- **[TI6]** El proyecto ha consistido íntegramente en concebir un servicio basado en tecnologías de red y computación móvil. Se ha diseñado una aplicación cliente sobre React Native que consume, mediante peticiones asíncronas HTTP/REST, tanto servicios en la nube como APIs externas de terceros y un microservicio propio de procesamiento. La integración de GPS, cámara, servicios meteorológicos y almacenamiento de objetos multimedia en una experiencia interactiva coherente ha reforzado directamente la capacidad de concebir sistemas y servicios apoyados en Internet y en la computación móvil.
- **[SI6]** El diseño de la solución ha integrado de forma explícita aspectos que trascienden lo puramente técnico. En el plano ético y social, la orientación del sistema hacia el aprovechamiento del vestuario existente y la sostenibilidad ha condicionado decisiones de diseño concretas, hasta el punto de descartar deliberadamente la funcionalidad de *web scraping* de catálogos por resultar incoherente con el principio de reducción del consumo textil. En el plano económico, la elección de un modelo *Backend as a Service* y de un desarrollo multiplataforma responde a un análisis de costes, tiempos y viabilidad ajustado a los recursos del proyecto. Todo ello se ha abordado aplicando métodos propios de la ingeniería del software.
- **[SI1]** La solución integra distintas tecnologías de la información y las comunicaciones: una capa cliente, un backend gestionado como servicio, un microservicio de procesamiento y servicios externos; en un flujo de procesos coherente que resuelve de forma efectiva una necesidad real de información y organización del usuario. La orquestación asíncrona de estos componentes heterogéneos para digitalizar prendas, generar conjuntos y planificar coladas ha reforzado la capacidad de combinar TIC y procesos para satisfacer necesidades concretas de manera eficiente.
- **[SI2]** La determinación de los requisitos del sistema ha atendido a la seguridad y al cumplimiento normativo. Al gestionar datos de carácter personal como son el perfil, la geolocalización y las imágenes del usuario, se han incorporado mecanismos de seguridad a nivel de datos: autenticación mediante tokens JWT firmados y validados, y Políticas de Seguridad a Nivel de Fila (RLS) nativas de PostgreSQL que garantizan que cada usuario solo acceda a su propia información, previniendo vulnerabilidades de tipo IDOR. Asimismo, el borrado en cascada de toda la información asociada a un usuario dado de baja se ha diseñado atendiendo al cumplimiento de la normativa de protección de datos.
- **[SI4]** Aunque el carácter individual del proyecto limita el ejercicio pleno de esta competencia, el marco metodológico ha reproducido de forma controlada el enlace entre la comunidad técnica y la de gestión: los tutores han asumido el rol de *Product Owner*, validando requisitos y priorizando el valor de las funcionalidades, mientras que la au-

6. CONCLUSIONES

tora ha ejercido de puente traduciendo esas decisiones de gestión a soluciones técnicas. Por otro lado, la orientación al usuario final ha reforzado la vertiente formativa de la competencia, mediante un catálogo visual de símbolos de cuidado con descripciones accesibles para usuarios no familiarizados con la simbología textil y justificaciones razonadas que acompañan cada recomendación de almacenamiento.

El refuerzo conjunto de estas competencias constituye una de las aportaciones formativas más relevantes del trabajo.

6.3 Limitaciones

Un análisis crítico del proyecto exige reconocer sus limitaciones, que lejos de invalidar los resultados, delimitan con precisión su alcance y abren la vía a mejoras futuras.

La mayor limitación enfrentada tiene que ver con el reconocimiento de las etiquetas de cuidado. El sistema de reconocimiento óptico opera exclusivamente sobre el texto de la etiqueta, extrayendo con fiabilidad la composición textil, pero no es capaz de interpretar automáticamente los símbolos normalizados de cuidado, o sea, los pictogramas de lavado, secado y planchado. La lectura automática de esta simbología constituye un problema de visión por computador mucho más complejo que el reconocimiento de texto. Su resolución habría requerido el entrenamiento de modelos específicos sobre conjuntos de datos etiquetados, tarea que excede el alcance temporal y material de este trabajo. Esta limitación se ha resuelto mediante un mecanismo de selección asistida en el que el usuario indica los símbolos presentes en la etiqueta a partir de un catálogo visual, solución que garantiza la completitud de la información de cuidado sin comprometer la fiabilidad del sistema.

En segundo lugar, la arquitectura del microservicio de procesamiento, desplegado en un entorno local, resulta idónea para un contexto de desarrollo y validación académica, pero plantea limitaciones de disponibilidad y escalabilidad de cara a un despliegue en producción con múltiples usuarios concurrentes. De esta forma, la lógica de recomendación de conjuntos se sustenta en un motor de reglas que produce resultados coherentes y es plenamente trazable pero no incorpora aprendizaje a partir de las preferencias o el historial del usuario.

6.4 Líneas de trabajo futuro

Las limitaciones identificadas, junto con las posibilidades que abre la solución desarrollada, permiten trazar diversas líneas de trabajo futuro que darían continuidad al proyecto.

- La primera línea de evolución sería la incorporación del reconocimiento automático de los símbolos de cuidado mediante técnicas de visión por computador, que sustituiría la selección manual actual por la detección directa de los pictogramas a partir de la fotografía de la etiqueta. Esta mejora requeriría la construcción o adopción de un conjunto de datos de símbolos textiles y el entrenamiento de un modelo de detección

específico.

- La segunda línea de gran valor consiste en dotar al motor de recomendación de capacidades de aprendizaje, evolucionando desde el sistema de reglas actual hacia un modelo que incorpore las valoraciones, el historial de uso y las preferencias explícitas del usuario, ofreciendo así sugerencias progresivamente más personalizadas.
- La tercera línea, desde la perspectiva del despliegue, una línea de trabajo relevante sería la migración del microservicio de procesamiento a una infraestructura en la nube escalable, que garantizase su disponibilidad permanente y su capacidad para atender a múltiples usuarios de forma simultánea, habilitando así la transición de la aplicación desde un prototipo funcional hacia un producto desplegable a escala real.
- Por último, apuntar que uno de los objetivos secundarios planteados anteriormente ha sido descartado de forma deliberada. Este objetivo es la integración de un sistema de *web scraping* para analizar catálogos en línea y sugerir nuevas adquisiciones. Se ha descartado por ser una funcionalidad orientada a fomentar la compra de ropa nueva lo que va en contra con uno de los principios del proyecto, la reducción del consumo textil y el mejor aprovechamiento de prendas que el usuario ya posee. Por coherencia con esta motivación, se ha optado por no implementar dicha funcionalidad al sistema.

6.5 Conclusión final

En conjunto, *ArmarioVirtual* da respuesta a la brecha identificada sobre la problemática real de la gestión del vestuario, materializando una aplicación funcional que integra digitalización automatizada, recomendación contextual, planificación de coladas y optimización espacial. El proyecto no solo ha alcanzado los objetivos técnicos propuestos, sino que ha constituido un ejercicio integral de ingeniería del software, desde la concepción teórica hasta la validación funcional, consolidando las competencias adquiridas a lo largo de la titulación y sentando una base sólida sobre la que construir las mejoras futuras identificadas.

ANEXOS

Anexo A

Anexo A

A.1 Declaración de uso de IA

A lo largo del desarrollo de este TFG se han utilizado herramientas de Inteligencia Artificial generativa como apoyo en distintas fases del proyecto. Este anexo tiene como objetivo declarar de forma transparente el uso de dichas herramientas, indicando cuándo se han empleado y con qué propósito, respetando en todo momento los principios éticos, académicos y de autoría del trabajo.

En ningún caso estas herramientas han sustituido el criterio propio ni la toma de decisiones técnicas y de diseño, que corresponden en su totalidad a la autora. A continuación se detallan las herramientas empleadas y la justificación de su uso.

- Claude (Anthropic). Esta herramienta se ha utilizado como apoyo en varias fases del proyecto. En el ámbito del desarrollo, se empleó para resolver problemas de codificación, depurar errores y contrastar distintos enfoques a la hora de implementar las funcionalidades de la aplicación. Asimismo, se utilizó como ayuda en la búsqueda y localización de bibliografía relevante para el marco teórico del proyecto.
- Gemini (Google). Esta herramienta se ha empleado principalmente como apoyo en la búsqueda de documentación y artículos de interés, con el fin de profundizar en la comprensión de los conceptos técnicos relacionados con el desarrollo de aplicaciones móviles multiplataforma, el procesamiento de imágenes y las tecnologías empleadas en el proyecto. También se utilizó de forma puntual como apoyo durante la resolución de dudas de codificación.

Cabe destacar que, en lo relativo a la búsqueda bibliográfica, las herramientas de IA se emplearon únicamente para agilizar la detección de fuentes relevantes. En ningún caso se incorporaron citas automáticas ni textos generados directamente al documento: todas las fuentes citadas fueron posteriormente localizadas, verificadas y contrastadas de forma manual por la autora.

A.2 Repositorio del Proyecto

El código fuente completo del proyecto ArmarioVirtual está disponible en el siguiente repositorio de GitHub:

<https://github.com/victorialpz/ArmarioVirtual.VictoriaLopezTFG.git>

A.2.1 Contenido del Repositorio

El repositorio incluye:

- Código de la aplicación móvil (React Native + Expo): aplicación multiplataforma para iOS y Android con toda la lógica de interfaz de usuario y gestión de estado.
- Configuración de Supabase: esquema relacional de la base de datos, políticas de RLS y scripts de inicialización.
- Microservicio de procesamiento de imágenes (Python + FastAPI): servicio local para eliminación de fondos con rembg y reconocimiento óptico de caracteres con EasyOCR.
- Documentación técnica: guías de instalación, configuración de variables de entorno y requisitos del sistema.
- Historial completo de commits: refleja el desarrollo iterativo del proyecto con etiquetas para cada milestone.

El progreso del proyecto puede consultarse en el tablero de GitHub Projects disponible en:

<https://github.com/users/victorialpz/projects/4/views/1>

Aquí se documentan todos los sprints, tareas completadas y el estado actual del desarrollo.

A.2.2 Estructura de Ramas

La rama main contiene el código estable y completamente funcional. Las ramas de desarrollo se utilizaron durante el proceso de implementación de cada módulo (feature/digitalización, feature/lavandería, etc.). La rama release contiene el código listo para ser entregado.

A.2.3 Ejecución Local

Para clonar el repositorio y ejecutar el proyecto en tu entorno local, consulta el archivo README.md en la raíz del repositorio.

Ahí se encuentran las instrucciones detalladas de instalación, dependencias requeridas y pasos de configuración necesarios para poner en marcha tanto la aplicación móvil como el microservicio de procesamiento.

Referencias

- [1] Marks & Spencer. Estudio sobre el tiempo dedicado a la elección diaria de vestuario. Sondeo a 2000 adultos, recogido en Complex Style, 2016. <https://www.complex.com/style/a/erica-euse/men-spend-four-months-of-lives-deciding-what-to-wear>.
- [2] European Environment Agency. Textiles in europe’s circular economy. Briefing no. 6/2019, European Environment Agency, 2019. <https://www.eea.europa.eu/publications/textiles-in-europes-circular-economy>.
- [3] Roy F. Baumeister, Ellen Bratslavsky, Mark Muraven, and Dianne M. Tice. Ego depletion: Is the active self a limited resource? *Journal of Personality and Social Psychology*, 74(5):1252–1265, 1998. <https://doi.org/10.1037/0022-3514.74.5.1252>.
- [4] Craig Bloem. This 1 unusual habit helped make mark zuckerberg, steve jobs, and dr. dre successful. Inc. Magazine, 2018. <https://www.inc.com/craig-bloem/this-1-unusual-habit-helped-make-mark-zuckerberg-steve-jobs-dr-dre-successful.html>.
- [5] Kirsi Niinimäki, Greg Peters, Helena Dahlbo, Patsy Perry, Timo Rissanen, and Alison Gwilt. The environmental price of fast fashion. *Nature Reviews Earth & Environment*, 1(4):189–200, 2020. <https://doi.org/10.1038/s43017-020-0039-9>.
- [6] Ellen MacArthur Foundation. A new textiles economy: Redesigning fashion’s future. Technical report, Ellen MacArthur Foundation, 2017. <https://www.ellenmacarthurfoundation.org/a-new-textiles-economy>.
- [7] Gediminas Adomavicius and Alexander Tuzhilin. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6):734–749, 2005. <https://doi.org/10.1109/TKDE.2005.99>.
- [8] Gediminas Adomavicius, Konstantin Bauman, Alexander Tuzhilin, and Moshe Unger. Context-aware recommender systems: From foundations to recent developments. In Francesco Ricci, Lior Rokach, and Bracha Shapira, editors, *Recommender Systems*

Handbook, pages 211–250. Springer, New York, NY, 3 edition, 2022. https://doi.org/10.1007/978-1-0716-2197-4_6.

- [9] Johannes Itten. *The Elements of Color: A Treatise on the Color System of Johannes Itten*. Van Nostrand Reinhold, New York, 1970.
- [10] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, volume 25, pages 1097–1105, 2012.
- [11] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. <https://doi.org/10.1109/5.726791>.
- [12] Xuebin Qin, Hang Dai, Xiaobin Hu, Deng-Ping Fan, Ling Shao, and Luc Van Gool. Highly accurate dichotomous image segmentation. In *European Conference on Computer Vision (ECCV)*, pages 38–56. Springer, 2022. https://doi.org/10.1007/978-3-031-19815-1_3.
- [13] Baoguang Shi, Xiang Bai, and Cong Yao. An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 39(11):2298–2304, 2017. <https://doi.org/10.1109/TPAMI.2016.2646371>.
- [14] Ken Schwaber and Jeff Sutherland. The scrum guide: The definitive guide to scrum: The rules of the game, nov 2020. <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>.
- [15] Eclipse Foundation. Openup: Open unified process. Eclipse Process Framework (EPF), 2012. https://archive.eclipse.org/epf/downloads/OpenUP/published/openup_published.1.5.1.5_20121212/openup/.
- [16] Per Kroll and Bruce MacIsaac. *Agility and Discipline Made Easy: Practices from OpenUP and RUP*. The Addison-Wesley Object Technology Series. Addison-Wesley Professional, 2006.
- [17] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2 edition, 2014. <https://git-scm.com/book/en/v2>.
- [18] Ricardo Pérez del Castillo. *Ingeniería del Software en Acción: De la velocidad, a la excelencia*. Publicación Independiente, 2 edition, 2025. <https://zenodo.org/doi/10.5281/zenodo.13709635>.

- [19] Andreas Biørn-Hansen, Christoph Rieger, Tor-Morten Grønli, Tim A. Majchrzak, and Gheorghe Ghinea. An empirical investigation of performance overhead in cross-platform mobile development frameworks. *Empirical Software Engineering*, 25(4):2997–3040, 2020. <https://doi.org/10.1007/s10664-020-09827-6>.
- [20] Akshat Paul and Abhishek Nalwaya. *React Native for Mobile Development: Resolve to Build React Native Apps*. Apress, Berkeley, CA, 2 edition, 2019. <https://doi.org/10.1007/978-1-4842-4454-8>.
- [21] Google. Better performance through threading. Android Developers — App Quality Guides, 2024. <https://developer.android.com/topic/performance/threads>.
- [22] Paul Castro, Vatche Ishakian, Vinod Muthusamy, and Aleksander Slominski. The rise of serverless computing. *Communications of the ACM*, 62(12):44–54, 2019. <https://doi.org/10.1145/3368454>.
- [23] Ramez Elmasri and Shamkant B. Navathe. *Fundamentals of Database Systems*. Pearson, 7 edition, 2015.
- [24] Michael B. Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force (IETF), 2015. <https://datatracker.ietf.org/doc/html/rfc7519>.
- [25] João Carreira, Pedro Fonseca, Alexey Tumanov, Andrew Zhang, and Randy Katz. Cirrus: A serverless framework for end-to-end ml workflows. In *Proceedings of the ACM Symposium on Cloud Computing (SoCC '19)*, pages 13–24, 2019. <https://doi.org/10.1145/3357223.3362711>.
- [26] Elisa Bertino and Ravi Sandhu. Database security — concepts, approaches, and challenges. *IEEE Transactions on Dependable and Secure Computing*, 2(1):2–19, 2005. <https://doi.org/10.1109/TDSC.2005.9>.

Este documento fue editado y tipografiado con $\text{\LaTeX}$
empleando la clase **giita-tfg** que se puede encontrar en:

<https://github.com/anarubioruiz/giita-tfg>

La clase **giita-tfg** está basada en la clase **esi-tfg**:

<https://github.com/UCLM-ESI/esi-tfg>

[respeta esta atribución de autoría]

